“Machine Learning & Synthetic Data Generation ”

Internship Based Project Report Submitted

To

Chhattisgarh Swami Vivekanand Technical University, Bhilai

for

Completion of 5th Semester Internship of

the degree of

BACHELOR OF TECHNOLOGY (HONORS)

In

[Artificial Intelligence]

(COMPUTER SCIENCE & ENGINEERING)

By

DURGESH KUMAR DEWANGAN

(Roll No.- **300012723018,** Enroll No. – **CC7457**)

B.Tech.(Hon) 5th Semester

Semester Internship ID : 0511/Intern

Under the Guidance of **Mr. Vachaspathi Madabhusnam**

(Associate ML Engineer at testAIng.com)

Department of Computer Science &Engineering

University Teaching Department (UTD)

Chhattisgarh Swami Vivekanand Technical University, Bhilai

Session: 2025-2026

DEPARTMENT OF COMPUTER SCIENCE &ENGINEERING

UNIVERSITY TEACHING DEPARTMENT

CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI

DECLARATION BY THE CANDIDATE

I, Durgesh Kumar Dewangan, hereby declare that the project work entitled "**Machine Learning Lifecycle Implementation with Synthetic Data Generation and TSTR Evaluation**"submitted in partial fulfillment of the requirements for the **Bachelor of Technology (Honours) in Computer Science and Engineering (Artificial Intelligence)** during the **5th Semester**, is the outcome of my own work carried out under the supervision of the faculty of **the Department of Computer Science and Engineering, Chhattisgarh Swami Vivekanand Technical University (CSVTU), Bhilai, Chhattisgarh.**

I further declare that:

The results, analyses, discussions, and conclusions presented in this report are based on the experiments, research work, and literature reviewed and performed by me.

This report has not been submitted, either wholly or partially, to any other university, institute, or authority for the award of any degree, diploma, or recognition.

Wherever contributions of others are involved in my work, every effort has been made to give due credit and proper acknowledgement.

I am fully responsible for the originality, correctness, and authenticity of the contents of this report.

I hereby grant the Department and University the right to use this work for academic, research, and reference purpose

DURGESH KUMAR DEWANGAN

Roll No. - 300012723018

Enrollment No. – CC745

DEPARTMENT OF COMPUTER SCIENCE &ENGINEERING
UNIVERSITY TEACHING DEPARTMENT
CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project, titled **" Machine Learning Lifecycle Implementation with Synthetic Data Generation and TSTR Evaluation ".**

Firstly, I would like toI express my sincere appreciation and deep respect to my project guide **Mr. Vachaspathi Madabhusnam(Associate ML engineer at testing.com)** whose constant support, insightful feedback, and valuable guidance have been instrumental in shaping this project. Their encouragement throughout the research and implementation activities enabled me to overcome challenges effectively.
I am also thankful to all the faculty members and technical staff of the department for their cooperation, constructive suggestions, and motivation during the development of this project.
My sincere thanks also go to **TestAIng Solutions Pvt. Ltd.** and the **AiEnsured team** for offering a technically rich platform and meaningful challenges that allowed me to work at the intersection of cutting-edge Large Language Models and Explainable AI.

I would also like to extend my deep appreciation to **Dr. Pankaj Kumar Mishra, Director, UTD, CSVTU Bhilai,**, whose leadership and vision have created an environment that fosters innovation, research, and academic excellence. Their commitment to academic development and industry collaboration provided the foundation for engaging in impactful research such as this.

DURGESH KUMAR DEWANGAN
Roll No. - 300012723018
Enrollment No. – CC745

ABSTRACT

Machine Learning (ML) has become a fundamental component in modern intelligent systems, enabling applications across various domains such as healthcare, finance, and automation. This project focuses on **the complete Machine Learning Life Cycle (MLLC**), covering essential stages including problem definition, data collection, preprocessing, exploratory data analysis, model selection, evaluation, deployment considerations, and continuous monitoring.

Alongside MLLC implementation, this work emphasizes the growing importance of **Synthetic Data Generation (SDG)** for enhancing dataset availability, maintaining privacy, and improving model robustness. The **SDV (Synthetic Data Vault)** framework was utilized for generating synthetic tabular data using **TVAE (Tabular Variational Autoencoder),** maintaining feature statistics and relationships present in real data. To validate the effectiveness of synthetic data, **TSTR (Train Synthetic, Test Real)** methodology was adopted using supervised learning models such **as K-Nearest Neighbors (KNN), Gradient Boosting, and XGBoost.**

Performance evaluation using metrics like accuracy, precision, recall, F1-score, and confusion matrix demonstrated that models trained on synthetic data achieved results comparable to those trained on real data, indicating high data fidelity and utility. The findings establish that advanced synthetic data generation techniques can serve as reliable alternatives in privacy-sensitive domains where real data access is restricted.

Overall, this project successfully demonstrates the practical implementation of the machine learning pipeline along with an empirical assessment of synthetic data quality, offering valuable insights into the integration of SDG approaches for scalable Artificial Intelligence solutions.

]

Table of Content

Section	Title	Page No.
1	1.1 Introduction to Machine Learning	1
	1.2. Supervised Machine Learning Models	2 - 9
	1.2 (A) Classification Models	
	1.2 (B) Regression Models:	
	1.3 Feature Categorisation	10
Task-1	Feature categorisation from the given datasets.	11 - 12
	1.4 Hyperparameters	13 - 15
2	Introduction to Machine Learning Life Cycle	16
Task-2	Implementation of MLLC in Jupyter Notebook & Explain How	17 - 21
3	Introduction of Synthetic Data Generation (SDG)	22 -23
	3.1 Data Augmentation	24 - 26
Task-3	Analysis for tabular data augmentation on datasets	27 - 28
	3.2 Test Data Generation (TDG)	29 - 30
Task-4	Research on TDM with features and GAN models used.	31 – 32
	3.3.Train Synthetic Test Real (TSTR) Methodology	33 - 34
Task-5	Implementation of the TSTR	35 -49
Task-6	Implementation of TSTR using techniques & hyperparameters.ipynb	50 – 51
Task-7	Implementation of TSTR using Gradient Boosting & KNN Comparison	52 - 53
Task-8	To perform an XGBoost classifier & analyse its accuracy	54 - 55
	Annexure I: Completion certificate	56

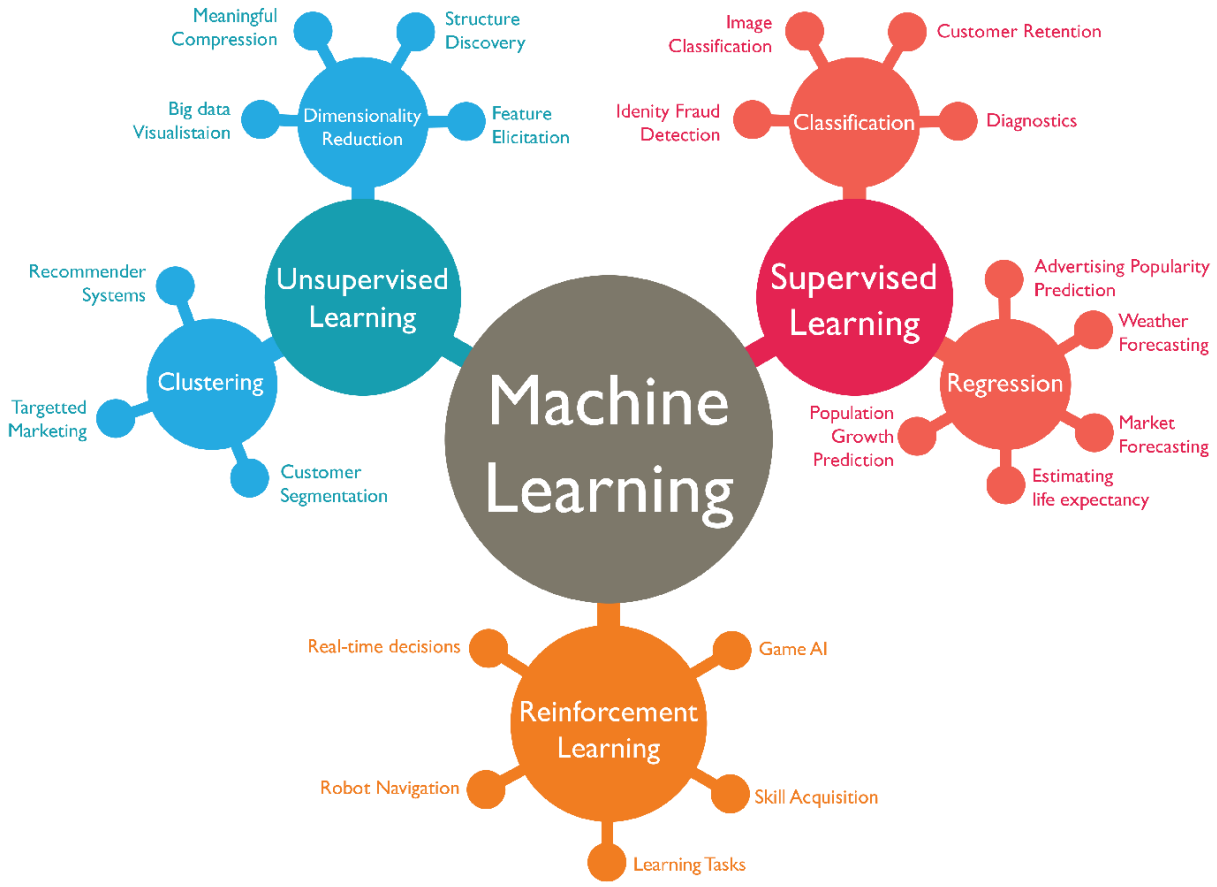
1.1 Introduction to Machine Learning

1.1 Introduction to Machine Learning (ML)

Machine Learning (ML) is a branch of artificial intelligence that enables computers to learn from data and improve their performance without being explicitly programmed. Instead of following fixed rules, ML systems identify patterns, relationships, and trends within data to make predictions or decisions. This makes ML highly effective for solving complex problems where traditional programming approaches are inefficient or impossible.

As the volume of digital data continues to grow, ML has become a crucial tool across industries. It powers everyday applications such as recommendation systems, speech recognition, fraud detection, medical diagnosis, and autonomous vehicles. ML algorithms are broadly categorized into supervised learning, unsupervised learning, and reinforcement learning—each designed for different types of tasks and data structures.

By allowing machines to adapt and learn from experience, machine learning plays a vital role in automating processes, enhancing decision-making, and creating intelligent systems. Its rapid development continues to shape the future of technology, making it an essential field for modern computing and innovation.



1.2.Supervised Machine Learning Models

Supervised Machine Learning is a core approach within machine learning where models are trained using labeled data. In this paradigm, each training example includes an input and a corresponding correct output, allowing the algorithm to learn a mapping from inputs to outputs. The goal is for the model to generalize this learned relationship so it can accurately predict outcomes for new, unseen data. Supervised learning is widely used because it offers precise and measurable results, especially in applications where historical data is available and clearly annotated.

Supervised models can be broadly divided into **classification** and **regression** algorithms. **Classification models** are used when the output is categorical, such as predicting whether an email is spam or not, identifying handwritten digits, or determining if a customer is likely to churn. Popular classification algorithms include **Logistic Regression**, **Decision Trees**, **Random Forests**, **Support Vector Machines (SVM)**, **K-Nearest Neighbors (KNN)**, and various **Neural Networks**.

On the other hand, **regression models** predict continuous numerical values. These are commonly used in forecasting house prices, estimating sales, or predicting temperature. Key regression models include **Linear Regression**, **Ridge and Lasso Regression**, **Support Vector Regression (SVR)**, and **Regression Trees**.

Training supervised models involves several steps: splitting data into training and testing sets, selecting an appropriate algorithm, optimizing model parameters, and evaluating performance using metrics such as accuracy, precision, recall, mean squared error, or R^2 score. The strength of supervised learning lies in its ability to leverage historical labeled datasets to produce highly reliable and interpretable predictions, making it essential for a wide range of real-world applications.

1.2. Supervised Machine Learning: Regression and Classification

Supervised Machine Learning is the type of ML that deals specifically with **labeled data**, meaning that each training example is paired with a known output. The goal of supervised learning is for a model to learn the relationship between inputs and outputs so it can accurately predict results for new, unseen data. This category of machine learning is especially useful when past data is well-organized and clearly annotated, enabling the model to recognize patterns and make informed predictions.

Supervised learning is divided into two main types: **regression** and **classification**.

1.2 (A) Classification Models

Classification models are a major category of supervised machine learning algorithms used to predict **categorical outcomes**. Instead of producing a continuous numeric value, classification models assign input data to one of several predefined classes or labels. These models learn from labeled training data—where each example includes both the input features and the correct class—to identify patterns that help distinguish different categories when new, unseen data is presented.

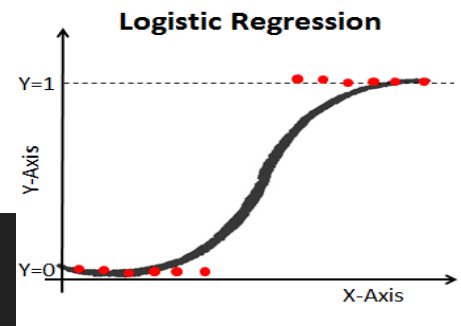
Classification tasks can be **binary**, where only two possible outcomes exist (such as “spam” vs. “not spam,” or “fraudulent” vs. “legitimate”), or **multiclass**, where the model must choose from three or more categories (e.g., types of flowers, handwritten digits, or product categories). In some advanced cases, classification can also be **multilabel**, in which an instance may belong to multiple classes simultaneously.

Several widely used classification algorithms include:

1. Logistic Regression

- **Definition:** A statistical model used to predict **binary outcomes** by estimating the probability of a class.
  Formula:

$$P(y=1|x) = \frac{1}{1+e^{-(\beta_0+\beta_1 x)}}$$



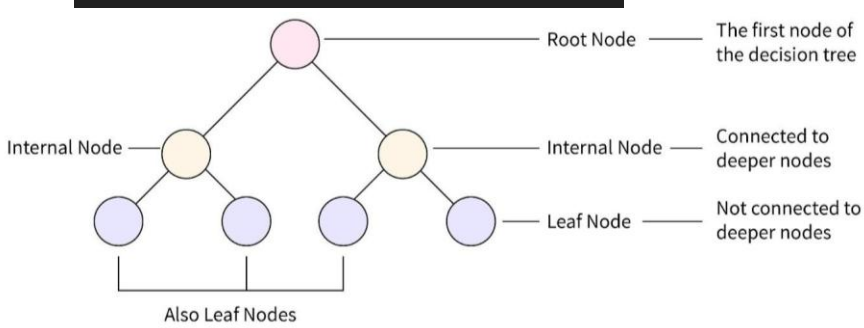
- **Use:** Spam detection, disease diagnosis, customer churn prediction.

---

2. Decision Tree Classifier

- **Definition:** Splits the dataset into branches based on feature values, assigning a class label at each leaf node.
  Formula (conceptual):

$$\hat{y}_R = \text{mode}(y_i \text{ in region } R)$$



- **Use:** Loan approval, customer segmentation, fraud detection.

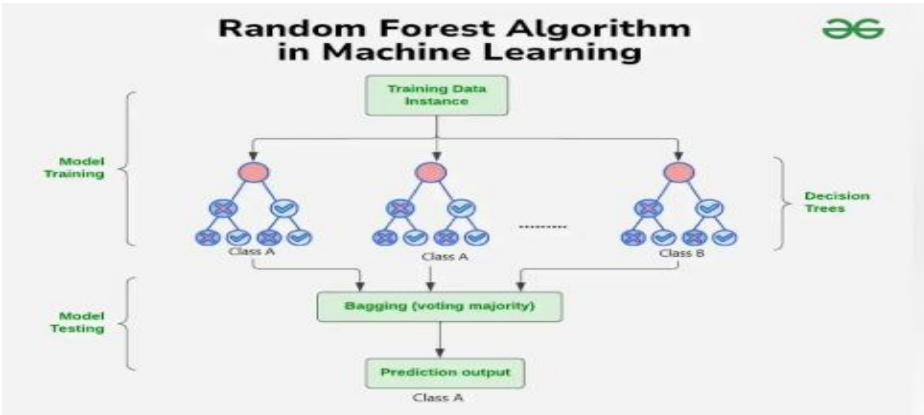
---

3. Random Forest Classifier

- **Definition:** Ensemble method combining multiple decision trees; assigns the **class predicted by majority of trees**.

  Formula:

$$\hat{y} = \text{majority_vote}(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T)$$



- **Use:** Image recognition, medical diagnosis, financial predictions.

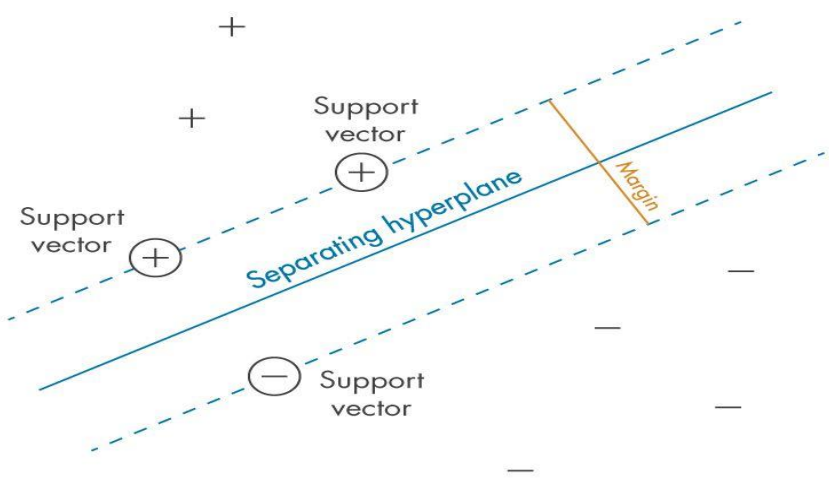
---

4. Support Vector Machine (SVM)

- **Definition:** Finds the **optimal hyperplane** that separates classes with the maximum margin.

  Formula:

$$\text{maximize } \frac{2}{\|w\|} \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1$$



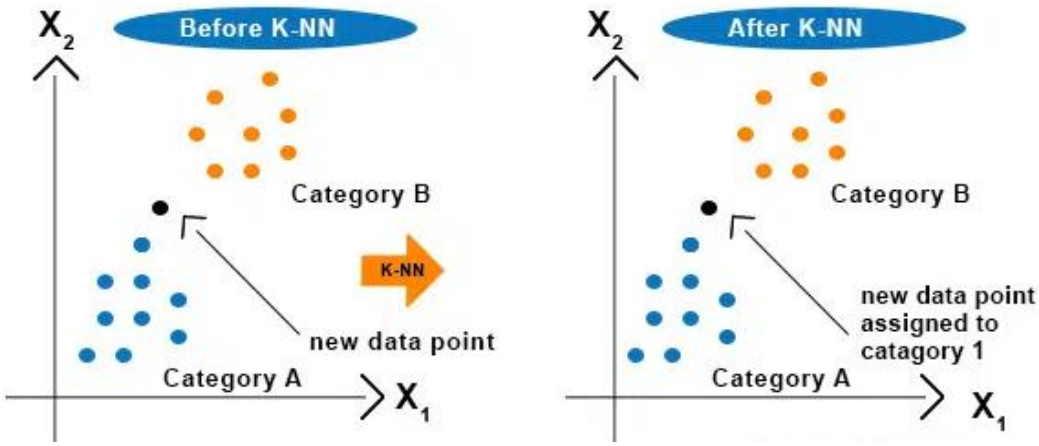
- **Use:** Text classification, face recognition, bioinformatics.

---

5. K-Nearest Neighbors (KNN)

- **Definition:** Classifies a data point based on the **majority class of its K nearest neighbors**.
  Formula (conceptual):

$$\hat{y} = \text{mode}(y_j), \quad j \in \text{K nearest neighbors of } x$$



- **Use:** Recommender systems, pattern recognition, anomaly detection.

---

6. Naive Bayes Classifier

- **Definition:** Probabilistic model using **Bayes' theorem** assuming feature independence.
  Formula:

$$P(C_k|x) = \frac{P(C_k)\prod_{i=1}^{n} P(x_i|C_k)}{P(x)}$$

- **Use:** Email spam detection, sentiment analysis, document classification.

---

7. Neural Networks (for Classification)

- **Definition:** Uses layers of interconnected nodes to **learn complex patterns** and classify data into multiple categories.
  Formula (sigmoid output for binary classification):

$$\hat{y} = \frac{1}{1+e^{-z}}, \quad z = \sum w_i x_i + b$$

- **Use:** Image recognition, speech recognition, autonomous systems..

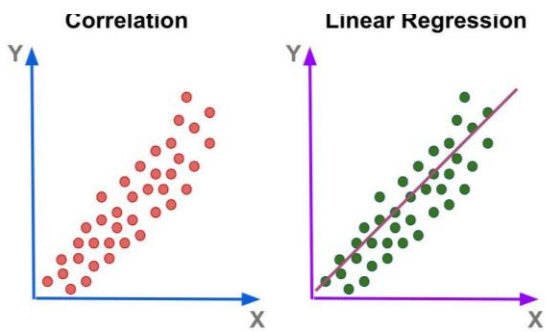
1.2 (B) Regression Models:

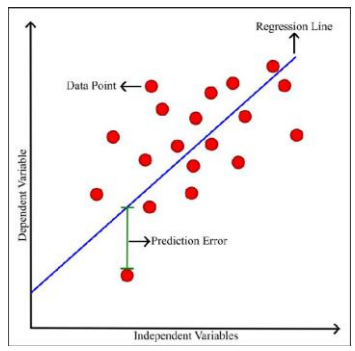
Regression is used when the output variable is continuous and numerical. The objective is to predict a real-valued quantity based on input features. For example, regression models can estimate house prices from attributes such as size, location, and age, or forecast future sales numbers using historical data. Common regression algorithms include Linear Regression, Polynomial Regression, and Support Vector Regression. These models evaluate patterns in data and minimize error to provide accurate numerical predictions.

1. Linear Regression

Definition: Predicts a **linear relationship** between a dependent variable and a single independent variable.
Formula:

$$y = \beta_0 + \beta_1 x + \epsilon$$





Use: House price prediction, sales forecasting.

---

2. Multiple Linear Regression

Definition: Extends linear regression to **multiple independent variables** to model linear relationships.
Formula:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \epsilon$$

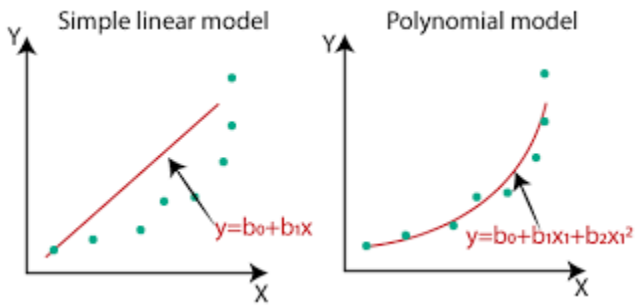
Use: Predicting car prices using features like mileage, age, and brand.

---

3. Polynomial Regression

Definition: Models **nonlinear relationships** by including powers of input features.
Formula:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_d x^d + \epsilon$$



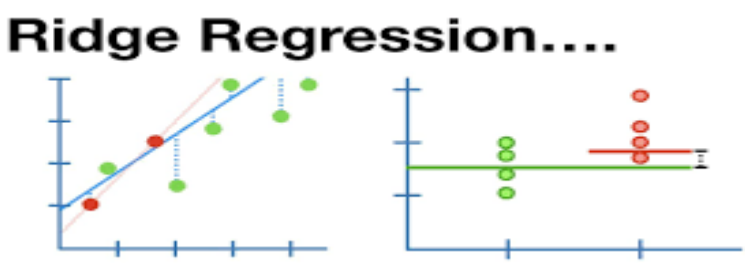
Use: Population growth, sales trends with seasonal patterns.

---

4. Ridge Regression

Definition: Linear regression with **L2 regularization** to prevent overfitting.
Formula:

$$\text{Minimize} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$



Use: Predicting financial metrics with correlated features.

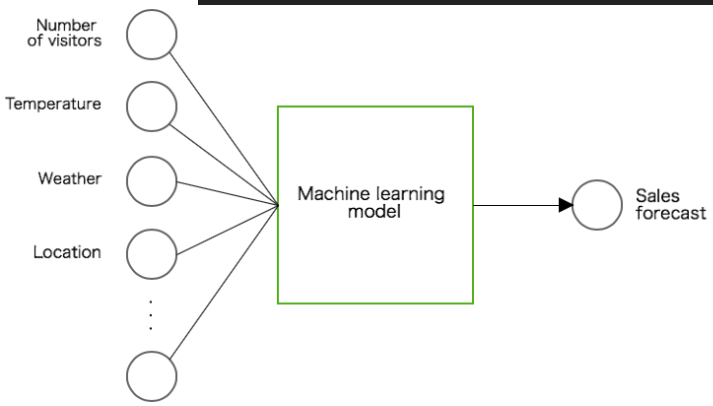
---

5. Lasso Regression

Definition: Linear regression with **L1 regularization**, also performs **feature selection**.

Formula:

$$\text{Minimize} \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$



Use: High-dimensional data, like genomics or text analysis.

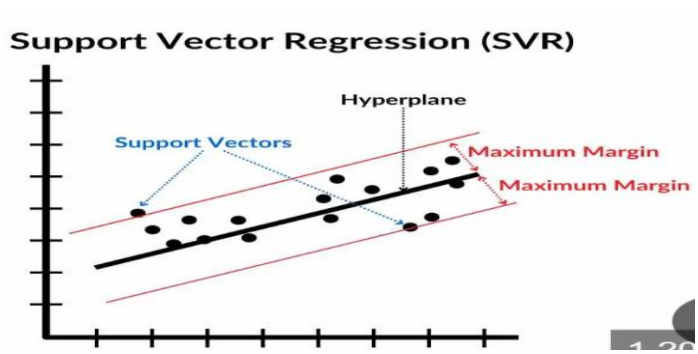
---

6. Support Vector Regression (SVR)

Definition: Predicts continuous values by fitting data within a **margin of tolerance** ((\epsilon)), can handle nonlinear relationships with kernels.

Formula:

$$\text{Minimize } \frac{1}{2}\|w\|^2 \quad \text{subject to } |y_i - (w \cdot x_i + b)| \leq \epsilon$$



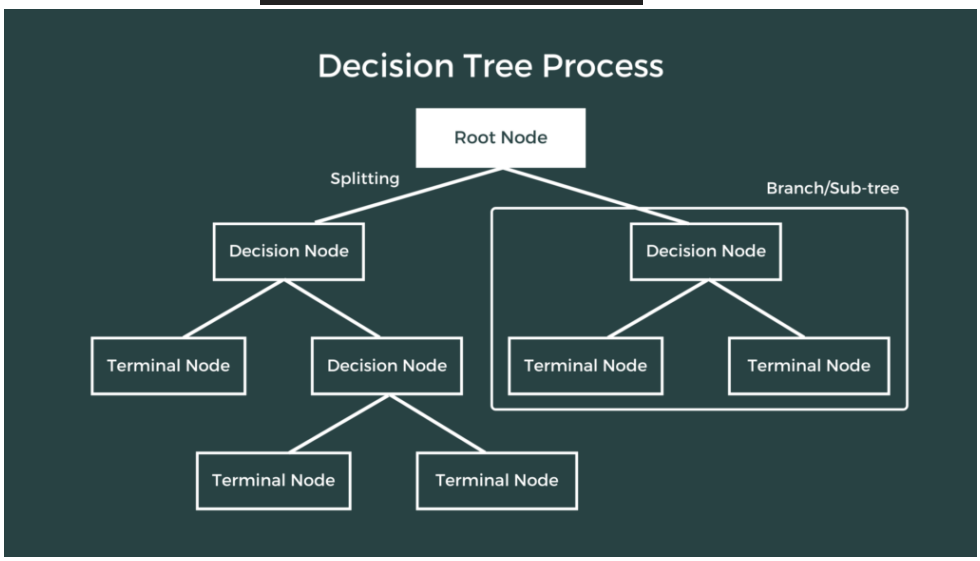
Use: Stock price prediction, energy demand forecasting.

---

7. Decision Tree Regression

Definition: Splits data into **regions** and predicts the output as the **average** of data points in each leaf.
Formula:

$$\hat{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$$

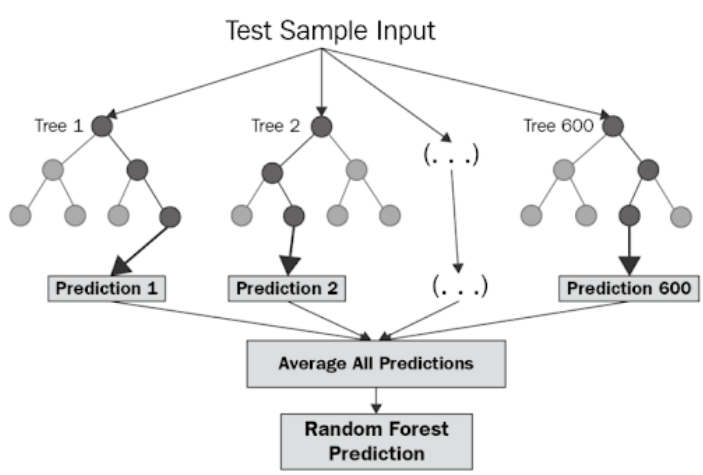


Use: Real estate pricing, customer spending prediction.

---

8. Random Forest Regression

Definition: An **ensemble of decision trees**, averages predictions from multiple trees for **robust predictions**.
Formula:



$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} \hat{y}_t$$

Use: Weather forecasting, loan default prediction, demand forecasting.

1.3 Feature Categorization

In machine learning, **features** (also called variables or attributes) are the individual measurable properties or characteristics of the data. Correctly identifying and categorizing features is critical because the type of feature determines which preprocessing techniques, algorithms, and evaluation metrics are appropriate.

Features can generally be categorized into the following types:

1. **Numerical Features (Quantitative Features)**
   - Represent measurable quantities and are expressed as numbers.
   - Can be **continuous** (any value in a range, e.g., temperature, weight, age) or **discrete** (countable values, e.g., number of students, number of products sold).
   - Common preprocessing: normalization, standardization, handling missing values.

2. **Categorical Features (Qualitative Features)**
   - Represent discrete categories or labels rather than numbers.
   - Can be **nominal** (no natural order, e.g., color, gender, product type) or **ordinal** (ordered categories, e.g., education level, satisfaction rating).
   - Common preprocessing: one-hot encoding, label encoding.

3. **Binary Features**
   - A special type of categorical feature with **two possible values** (0/1, True/False, Yes/No).
   - Often used in classification problems.

4. **Date and Time Features**
   - Represent timestamps or durations.
   - Can be decomposed into components like year, month, day, hour, or converted into numerical features such as elapsed time.

5. **Text Features**
   - Represent unstructured textual data.
   - Can be processed using techniques like bag-of-words, TF-IDF, or word embeddings to convert text into numerical vectors.

6. **Derived or Engineered Features**
   - Created from raw features to enhance model performance.
   - Examples: combining day and month into a “season” feature, calculating BMI from height and weight.

Importance: Feature categorization helps in **selecting appropriate algorithms, preprocessing steps, and feature transformations**, ultimately improving model accuracy and interpretability.

<u>Task</u> -1: Feature categorisation from the given datasets.

Loan_Prediction.csv

Features	DatatType	Feature type	Explain
CustomerID	Int64	NUM_INT_ID	ID is unique for each customer
Age	Int64	NUM_INT_COUNT	Discrete age no. for each customer
Gender	Int64	OBJ_STR_BINARY	Binary data in 0 & 1
AnnualIncome	Int64	NUM_INT_RATIO	Continues number
EmploymentStatus	Int64	OBJ_STR_BINARY	Binary data
LoanAmount	Int64	NUM_INT_RATIO	
LoanTermMonths	Int64	NUM_INT_COUNT	Discrete no. but countable
CreditScore	Int64	NUM_INT_RATIN G	Represent the score
DebtToIncomeRatio	float64	NUM_FLOAT_RA TIO	Showing the debt to income ratio.
PreviousLoans	Int64	NUM_INT_COUNT	Counting the past loan records
LoanDefault	Int64	OBJ_STR_BINARY	It is on binary response 0 & 1

Health_disease_prediction.csv

Feature	Data type	Feature type	Labeltype /Describe
PatientID	Int64	NUM_INT_ID	Unique ID for each patient
Age	Int64	NUM_INT_COUNT	Discrete Age
BMI	Int64	NUM_INT_COUNT	Discrete level
BloodPressure	Int64	NUM_INT_COUNT	Discrete BP level
SugarLevel	Int64	NUM_INT_COUNT	Discrete sugar level
HeartRate	Int64	NUM_INT_COUNT	Discrete heartrate
Disease	Int64	OBJ_STR_BINARY	Boolean

Churn_Modelling_Dataset.csv

Feature	**Data Type**	**Feature type**	**Labeltype /Describe**
RowNumber	Int64	NUM_INT_ID	Unique Continuous serial number
CustomerID	Int64	NUM_INT_ID_PII	Uniques id for each customer
Surname	object	OBJ_STR_TEXT	Name Surname of the each customer
Creditscore	Int64	NUM_INT_RATIN G	Score of each customer
Geography	object	OBJ_STR_CAT	Category for country of each customer
Gender	object	OBJ_STR_BINARY	Boolean value of customer in M and F
Age	Int64	NUM_INT_COUNT	Discrete age value of each person
Tenure	Int64	NUM_INT_COUNT	Discrete tensure
Balance	Float64	NUM_FLOAT_QT Y	Continues floating value
NumofProducts	Int64	NUM_INT_COUNT	Discrete number of products
HasCrCard	Int64	OBJ_STR_BINARY	Boolean calue in 0 and 1
IsActiveMember	Int64	OBJ_STR_BINARY	Boolean calue in 0 and 1
EstimatedSalary	Float64	NUM_FLOAT_QT Y	Continues floating value
Exited	object	OBJ_STR_BINARY	Boolean calue in 0 and 1

1.4 Hyperparameters

Definition:
Hyperparameters are parameters set **before training a model** that control the learning process and behavior of machine learning algorithms. They are **not learned from data** like model parameters (e.g., weights) but must be manually chosen or tuned to optimize performance. Correct hyperparameter selection can improve accuracy, prevent overfitting, and accelerate training.

1. Learning Rate (α)

Formula (Gradient Descent update):

$$\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Where, **θ** = model parameters, **J(θ)** = loss function, **α** = learning rate

Python Example (for Linear Regression using Gradient Descent):

```
theta = theta - learning_rate * gradient
```

---

2. Number of Epochs

Definition: Number of passes over the entire training dataset.
Effect: Too few → underfitting, too many → overfitting.

Python Example (using Keras):

```python
model.fit(X_train, y_train, epochs=50, batch_size=32)
```

---

3. Batch Size

Definition: Number of samples used to compute the gradient before updating model parameters.
Effect: Smaller batch → noisy updates, better generalization; larger batch → faster training.

Python Example (Keras):

```python
model.fit(X_train, y_train, batch_size=64, epochs=20)
```

---

4. Regularization Parameters (Ridge/Lasso)

Ridge (L2) Formula:

$$\text{Loss} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Lasso (L1) Formula:

$$\text{Loss} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

Python Example (Ridge Regression in scikit-learn):

```python
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)  # alpha = λ
model.fit(X_train, y_train)
```

---

5. Number of Trees / Depth (Decision Tree / Random Forest)

Definition: Controls model complexity. Maximum depth limits tree size; more trees improve ensemble performance.

Python Example (Random Forest):

```python
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(n_estimators=100, max_depth=10)
model.fit(X_train, y_train)
```

---

6. Kernel Type (SVM)

Formula (Linear SVM Decision Function):

$$f(x) = w \cdot x + b$$

Python Example:

```python
from sklearn.svm import SVC
model = SVC(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
```

---

7. Dropout Rate (Neural Networks)

Definition: Fraction of neurons randomly ignored during training to prevent overfitting.

Python Example (Keras):

```python
from tensorflow.keras.layers import Dropout
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))  # 30% dropout
```

Hyperparameter Tuning Methods

1. **Grid Search:** Exhaustively tests combinations.
2. **Random Search:** Randomly samples hyperparameters.
3. **Bayesian Optimization / AutoML:** Probabilistic optimization for efficient search.

Python Example (Grid Search):

```python
from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators': [50, 100, 150], 'max_depth': [5, 10, 15]}
grid = GridSearchCV(RandomForestRegressor(), param_grid, cv=5)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_
```

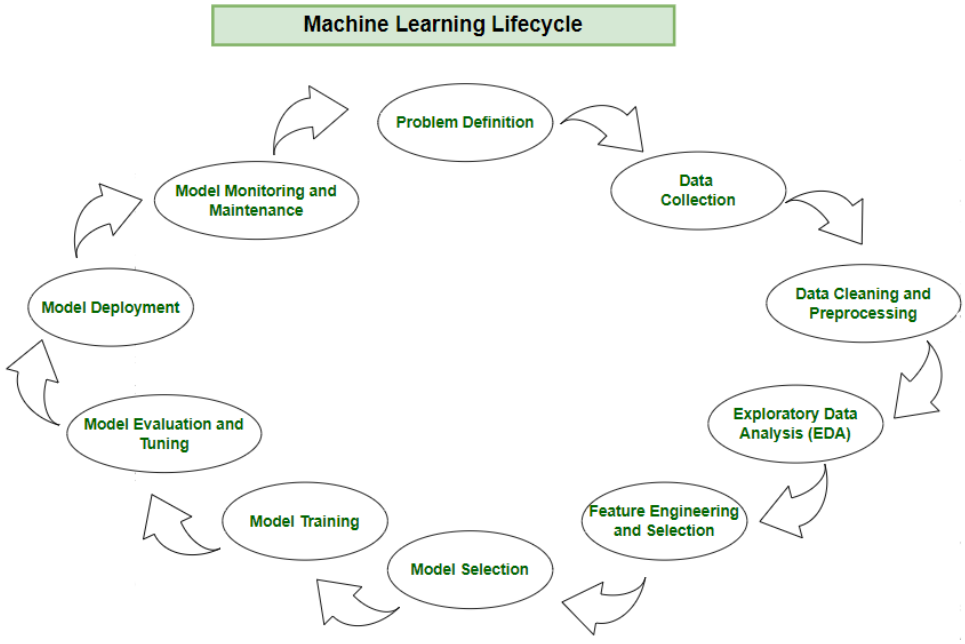
2. Introduction to Machine Learning Life Cycle

2. Machine Learning Life Cycle

Machine Learning Life Cycle (MLLC) – Definition:

The Machine Learning Life Cycle (MLLC) is a systematic framework that outlines the end-to-end process of developing, deploying, and maintaining machine learning models. It encompasses all stages from understanding and defining the problem, collecting and preprocessing relevant data, performing exploratory data analysis, engineering and selecting meaningful features, training and evaluating models, to deploying them in real-world applications and continuously monitoring their performance. The MLLC ensures that machine learning solutions are robust, accurate, and aligned with business or research objectives, providing a structured approach to manage the complexity of building effective AI systems while addressing challenges like data quality, model generalization, and evolving data patterns.

The main stages are:



Task-2: Implementation of MLLC in JupyterNotebook & Explain How

1. Problem Definition

Practical Execution:

- Understand the business objective or research question.
- Translate it into an ML problem type (classification, regression, clustering, recommendation).
- Define success metrics (e.g., accuracy > 90%, RMSE < 5).

  Example: Business goal: Reduce customer churn.
  ML problem: Binary classification predicting "churn" vs. "not churn."
  Success metric: F1-score or ROC-AUC > 0.8.

---

2. Data Collection

Practical Execution:

- Gather relevant data from multiple sources such as databases, APIs, logs, or external datasets.
- Combine and consolidate data for model use.

  Example: Collect customer profiles, transaction history, customer support tickets, and website interaction logs.import pandas as pd

```
Path = '/content/drive/MyDrive/Durgesh(testing)/Code (ML lifecycle)/EV_Data.csv'
Ev_adoption = pd.read_csv(Path)
Ev_adoption.head(410)
```

---

3. Data Preprocessing & Cleaning

Practical Execution:

- Handle missing values (imputation, deletion).
- Remove duplicates and irrelevant features.
- Encode categorical variables (one-hot or label encoding).Ev_adoption.isnull
- Normalize or standardize numerical features.

Example:

python

```python
from sklearn.preprocessing import StandardScaler
import pandas as pd

df = pd.read_csv("customer_data.csv")
df.fillna(df.mean(), inplace=True)
df = pd.get_dummies(df, columns=['gender','region'])
scaler = StandardScaler()
df[['age','income']] = scaler.fit_transform(df[['age','income']])
```

```python
Ev_adoption.isnull()
```

	Unnamed: 0.1	Unnamed: 0	ex party	year	EV Registrations	Total Vehicles	EV Share (%)	Stations	Total Charging Outlets	Level 1	...	personal	reducetax	regulate	worried	price_cents_per_kwh	gasoline_price_per_gallon	Total	Trucks	Trucks_Share	Party
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
403	False	False	False	False	False	False	False	False	False	False	...	True	True	True	True	False	True	True	True	True	True
404	False	False	False	False	False	False	False	False	False	False	...	True	True	True	True	False	True	True	True	True	True
405	False	False	False	False	False	False	False	False	False	False	...	True	True	True	True	False	True	True	True	True	True
406	False	False	False	False	False	False	False	False	False	False	...	True	True	True	True	False	True	True	True	True	True
407	False	False	False	False	False	False	False	False	False	False	...	True	True	True	True	False	True	True	True	True	True

408 rows × 36 columns

```python
Ev_adoption.isna().sum()
```

	0
Unnamed: 0.1	0
Unnamed: 0	0
ex party	0
year	0
EV Registrations	0
Total Vehicles	0
EV Share (%)	0
Stations	0
Total Charging Outlets	0
Level 1	0
Level 2	0
DC Fast	0
fuel_economy	102
Incentives	8
Number of Metro Organizing Committees	51
Population_20_64	0
Education_Bachelor	0
Labour_Force_Participation_Rate	0
Unemployment_Rate	0
Bachelor_Attainment	0
Per_Cap_Income	0
affectweather	102
devharm	102
discuss	102
exp	102
localofficials	102
personal	102
reducetax	102
regulate	102
worried	102
price_cents_per_kwh	0

- To remove the row and column with the missing values using the .dropna()
- Ev_adoption.dropna()

```
[64] Ev_adoption.dropna()
```

	Unnamed: 0.1	Unnamed: 0	ex party	year	EV Registrations	Total Vehicles	EV Share (%)	Stations	Total Charging Outlets	Level 1	...	personal	reducetax	regulate	worried	price_cents_per_kwh	gasoline_price_per_gallon	Total	Trucks	Trucks_Share	Party
0	0	0	Alabama	2023	13000	4835900	0.27	424	1096	35	...	39.06	62.16	69.01	54.95	11.47	2.742	5.446619e+06	3.397137e+06	62.37	Republican
1	1	1	Alaska	2023	2700	559800	0.48	65	124	3	...	43.28	65.38	71.61	62.49	21.41	3.594	6.809740e+05	5.175250e+05	76.00	Republican
2	2	2	Arizona	2023	89800	6529000	1.38	1198	3506	9	...	46.92	64.73	73.19	64.32	12.19	3.278	6.447062e+06	3.868118e+06	60.00	Democratic
3	3	3	Arkansas	2023	7100	2708300	0.26	334	833	3	...	39.08	63.39	68.58	56.21	9.73	2.760	3.338322e+06	2.291924e+06	68.65	Republican
4	4	4	California	2023	1256600	36850300	3.41	16381	49433	648	...	53.19	72.08	76.30	71.24	24.87	4.731	3.105733e+07	1.675788e+07	53.96	Democratic
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
301	301	301	Virginia	2018	9900	7694500	0.13	615	1446	95	...	46.07	69.88	78.22	65.58	9.48	2.460	7.604646e+06	4.106265e+06	54.00	Democratic
302	302	302	Washington	2018	30200	6644500	0.45	961	2573	171	...	45.90	68.73	77.29	66.46	8.00	3.461	7.152413e+06	3.928127e+06	54.92	Democratic
303	303	303	West Virginia	2018	200	1515200	0.01	89	212	9	...	34.35	54.03	71.69	53.55	8.72	2.532	1.693719e+06	1.069746e+06	63.16	Republican
304	304	304	Wisconsin	2018	3700	5512200	0.07	306	557	90	...	39.91	67.22	78.50	60.46	10.58	2.407	5.683061e+06	3.244241e+06	57.09	Democratic
305	305	305	Wyoming	2018	200	623600	0.03	45	121	4	...	34.15	51.72	71.15	55.00	8.09	2.577	8.370241e+05	6.004518e+05	71.74	Republican

250 rows × 36 columns

-
-
- For removing the duplicate data from the dataset we can use .drop_duplicate()
- Ev_adoption.drop_duplicates()
-

---

4. Exploratory Data Analysis (EDA)

Practical Execution:

- Visualize data distributions and relationships.
- Detect patterns, outliers, or correlations.
- Identify feature importance.

Example:

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.corr(), annot=True)
plt.show()
sns.boxplot(x='churn', y='age', data=df)
```

---

5. Feature Engineering & Selection

Practical Execution:

- Create new features from existing ones (e.g., tenure × monthly charges).
- Select important features using correlation analysis, Lasso, or tree-based methods.

Example:

```python
df['avg_transaction_per_month'] = df['total_transactions'] / df['tenure_months']
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier

sel_model = RandomForestClassifier()
sel_model.fit(X_train, y_train)
selected_features = X_train.columns[sel_model.feature_importances_ > 0.01]
```

---

6. Model Selection & Training

Practical Execution:

- Split data into **training and testing** sets.
- Train multiple models (e.g., Logistic Regression, Random Forest, XGBoost).
- Tune **hyperparameters** for optimal performance.

Example:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(X[selected_features], y, test_size=0.2)
model = RandomForestClassifier(n_estimators=100, max_depth=10)
model.fit(X_train, y_train)
```

---

7. Model Evaluation

Practical Execution:

- Use metrics suited for the problem (classification: accuracy, precision, recall, F1-score; regression: RMSE, R^2).

- Perform cross-validation to test generalization.

  Example:

```
from sklearn.metrics import classification_report, roc_auc_score

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, model.predict_proba(X_test)[:,1]))
```

---

8. Model Deployment

Practical Execution:

- Deploy the model to production via **APIs, web services, or dashboards**.
- Integrate with applications for real-time or batch predictions.

  Example:

```
import joblib
joblib.dump(model, 'churn_model.pkl')  # Save model
# Load in production: model = joblib.load('churn_model.pkl')
```

- Use frameworks like Flask, FastAPI, or cloud services (AWS Sagemaker, Azure ML).

---

9. Monitoring & Maintenance

Practical Execution:

- Continuously track model performance (accuracy, drift).
- Log predictions and feedback for retraining.
- Update the model periodically to handle **data drift** or **concept drift**.

  Example: Set up dashboards monitoring F1-score, prediction distribution.
  Retrain the model monthly with new customer data.

Work Flow (Practical)

Business Problem → Data Collection → Data Cleaning → EDA → Feature Engineering → Model Training → Evaluation → Deployment → Monitoring

---

3. Synthetic Data Generation (SDG)

Definition:
Synthetic Data Generation (SDG) is the process of **creating artificial data** that mimics the statistical properties, patterns, and structure of real-world datasets. Unlike data augmentation, which modifies existing data, synthetic data is **entirely generated**. SDG is widely used in machine learning, testing, and simulation when real data is **limited, sensitive, or costly** to collect.

Purpose of Synthetic Data

1. **Privacy Preservation:**
   - Avoids exposing sensitive or personally identifiable information (PII) while maintaining data utility.

2. **Data Scarcity Mitigation:**
   - Useful when real-world data is insufficient to train robust ML models.

3. **Testing and Validation:**
   - Enables testing ML models or software systems under **rare, edge-case, or extreme scenarios**.

4. **Model Training and Benchmarking:**
   - Provides large-scale datasets for training, hyperparameter tuning, and benchmarking models.

Techniques for Synthetic Data Generation

1. Random Data Synthesis

- Generates random values within specified ranges or distributions.
- **Use case:** Simple test datasets or simulations.
  Python Example:

```python
import numpy as np
import pandas as pd

synthetic_data = pd.DataFrame({
    'age': np.random.randint(18, 70, 100),
    'income': np.random.randint(20000, 120000, 100),
    'gender': np.random.choice(['Male', 'Female'], 100)
})
print(synthetic_data.head())
```

---

2. Rule-Based or Constraint-Based Generation

- Generates data following **business rules or constraints**.
- Ensures logical relationships between features.

Example: total = quantity * price

```
quantity = np.random.randint(1, 20, 100)
price = np.random.randint(50, 500, 100)
total = quantity * price

synthetic_data = pd.DataFrame({'quantity': quantity, 'price': price, 'total': total})
```

---

3. Probabilistic / Statistical Methods

- Samples from known distributions (normal, uniform, Poisson, etc.) to generate realistic data.

Example: Generate normally distributed income data

```
income = np.random.normal(loc=50000, scale=15000, size=100)
```

---

4. Generative Models (Advanced)

- Use machine learning models to generate realistic synthetic data.
- **Techniques:**
  - **GANs (Generative Adversarial Networks):** Generate images, tabular, or time-series data.
  - **VAEs (Variational Autoencoders):** Learn data distribution and generate similar samples.

Python Example (Tabular Data using scikit-learn's make_classification):

from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=5, n_informative=3, n_classes=2, random_state=42)

synthetic_data = pd.DataFrame(X, columns=['feature1','feature2','feature3','feature4','feature5'])

```
synthetic_data['target'] = y
```

Benefits of Synthetic Data

- Overcomes **data privacy concerns**.
- Reduces dependence on expensive or hard-to-collect datasets.
- Enables testing of **rare scenarios** or **imbalanced classes**.
- Facilitates **rapid prototyping** and ML model development.

3.1 Data Augmentation

Definition:
Data Augmentation is the process of **artificially increasing the size and diversity of a dataset** by creating modified versions of existing data. It is commonly used in machine learning, especially in **image, text, and time-series datasets**, to improve model generalization and robustness.

Purpose of Data Augmentation

1. **Increase Dataset Size:**
   - Helps when the available dataset is small, reducing overfitting.

2. **Improve Model Generalization:**
   - Exposes the model to varied inputs, making it more robust to unseen data.

3. **Handle Class Imbalance:**
   - Synthetic data can be generated for underrepresented classes to balance the dataset.

4. **Simulate Real-World Variations:**
   - For example, rotation, noise, or distortion in images, or synonym replacement in text.

Common Data Augmentation Techniques

1. Image Data Augmentation

- Rotation, flipping, scaling, cropping, translation.
- Adding noise, changing brightness/contrast.

Python Example (using Keras):

```python
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    zoom_range=0.1
)

# Example: augmenting a single image
import numpy as np
from tensorflow.keras.preprocessing import image

img = image.load_img('sample.jpg', target_size=(128,128))
x = image.img_to_array(img)
x = x.reshape((1,) + x.shape)

aug_iter = datagen.flow(x, batch_size=1)
augmented_image = next(aug_iter)[0].astype('uint8')
```

---

2. Text Data Augmentation

- Synonym replacement, random insertion, deletion, and swapping.
- Back-translation (translate text to another language and back).

Python Example (using nlpaug):

```python
import nlpaug.augmenter.word as naw

text = "Machine learning is fun"
aug = naw.SynonymAug(aug_src='wordnet')
augmented_text = aug.augment(text)
print(augmented_text)
```

---

3. Tabular / Structured Data Augmentation

- SMOTE (Synthetic Minority Oversampling Technique) for balancing classes.
- Adding Gaussian noise to numeric features.

Python Example (SMOTE):

```python
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy='minority')
X_res, y_res = smote.fit_resample(X_train, y_train)
```

---

4. Time-Series Data Augmentation

- Time warping, scaling, jittering, and window slicing.

Python Example:

```
# Adding Gaussian noise to a time-series
import numpy as np
time_series = np.array([1,2,3,4,5,6,7])
augmented_series = time_series + np.random.normal(0, 0.1, size=time_series.shape)
```

Task-3 : Analysis for tabular data augmentation on datasets

DATE : 26/05/2025

In Pandas library:

Example Code:

```
Import pandas as pd

Path = ‘  //.csv’

df = pd.read_csv(Path)

df.head()
```

Function, variable, etc	Explain
Import pandas as pd	Importing pandas library to use the dataframe
Path = ‘ //.csv’	string variable (Path), holds location of file
df = pd.read_csv(Path)	using the pandas read_csv() function to read the csv file and store the result in df variable.
df.head()	Function use to show top records of file or datasets as per user want no. of records

- For each dataset, performed:
  1) Per-feature type identification (int, float, object)

  df.dtypes It is used to get the information. It shows the data type for every column. Where the df is the dataframe and dtypes return the datatype of each column.

  2) Descriptive statistics analysis :
     - Count- how many value are in the datasets.
     - Min/Max- find the smallest or largest value .
     - Mean- Average value of the datasets
     - Standard Deviation(std)- Spread out a set of data is from its avg.

  df.describe(): It is used to generate descriptive statistics of dataframe(df).

  3) Value range classification:
     i. Features with values 0-1 for normalization
     ii. Features with values > 1 for refinement

Example code:

```
def classify_ranges(series):
```

```
    if pd.api.types.is_numeric_dtype(series):
        min = series.min()
        max = series.max()
        if min == 1 and max == 1:
            return "1 -> refine"
        elif 0 <= min and max <= 1:
            return "0–1 - normalize"
        else:
            return f"{min}-{max} -> consider scaling"
    else:
        return "non-numeric -> consider encoding"
print('value range classification:')
df.apply(classify_ranges)
```

	Function	Explaination
	df function_name(series):	define own function name that take single column from dataframe as input
	if pd.api.types.is_numeric_dtype():	(if)check the condition & pd.api.types.is_numeric_dtype() is pandas utility function to detect numeric datatype
	min = series.min() max = series.max()	Use to calculate the minimum and maximum values in the column
	if min == 1 and max == 1: return "1 -> refine"	if value are exactly 1 , it returns 1 -> refine
	elif 0 <= min and max <= 1: return "0–1 -> normalize"	if value b/w 0 &1 , it returns 0-1 -> normalize
	else: return f"{min}-{max} -> consider scaling"	If value is not in range like 0-1 then it means range scaling needed.
	else: return "non-numeric -> consider encoding	if it is not numeric then return the message only.

3.2 Test Data Generation (TDG)

Definition:
Test Data Generation (TDG) is the process of creating **synthetic or simulated datasets** that are used to evaluate machine learning models or software systems. TDG ensures models and systems are tested thoroughly, including under conditions that may be rare, sensitive, or not well-represented in real data.

Purpose of TDG

1. **Model Evaluation:**
   - Allows testing of ML models on controlled datasets to assess performance metrics like accuracy, precision, recall, and robustness.

2. **Testing Edge Cases / Rare Scenarios:**
   - Helps simulate unusual or extreme conditions that may not appear frequently in real data but are critical for reliability.

3. **Increasing Data Coverage:**
   - Useful when **real data is limited, incomplete, or sensitive**, such as healthcare or financial datasets.

Techniques for Test Data Generation

1. **Random Data Synthesis:**
   - Generates data by randomly sampling from possible values or distributions.
- Example: Random integers, strings, or dates for testing input validatio

```
import numpy as np
import pandas as pd

# Generate 100 random test records
np.random.seed(42)
random_data = pd.DataFrame({
    'age': np.random.randint(18, 65, size=100),           # Random ages between 18-64
    'income': np.random.randint(20000, 120000, size=100), # Random income
    'gender': np.random.choice(['Male', 'Female'], size=100)  # Random gender
})

print(random_data.head())
```

2. **Rule-Based Generation:**
   - Creates data following specific **predefined rules** or business logic.
   - Example: Generating test customer ages between 18 and 65, or phone numbers in valid formats.

```python
# Rule: Age between 21-60, income >= 30000 for working adults
rule_based_data = pd.DataFrame({
    'age': np.random.randint(21, 61, size=100),
    'income': np.random.randint(30000, 120000, size=100),
    'employment_status': ['Employed' if x >= 30000 else 'Unemployed' for x in np.random.randint(20000, 120000, size=100)]
})

print(rule_based_data.head())
```

3. **Constraint-Based Generation:**
   - Produces data that satisfies **complex constraints** or relationships between variables.
   - Example: Ensuring end_date > start_date in event datasets, or total = price × quantity.

```python
# Constraint: total_purchase = quantity * price
quantity = np.random.randint(1, 20, size=100)
price = np.random.randint(50, 500, size=100)
total_purchase = quantity * price

constraint_data = pd.DataFrame({
    'quantity': quantity,
    'price': price,
    'total_purchase': total_purchase
})

print(constraint_data.head())
```

4. **Generative Models:**
   - Uses machine learning models (like **GANs, VAEs**) to generate **realistic synthetic data** resembling the original dataset.
   - Example: Generating synthetic medical images or customer transaction records while preserving statistical properties.

```python
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=5, n_informative=3, n_classes=2, random_state=42)
synthetic_data = pd.DataFrame(X, columns=['feature1', 'feature2', 'feature3', 'feature4', 'feature5'])
synthetic_data['target'] = y

print(synthetic_data.head())
```

Task -4: Research on TDM with features and GAN models used.

Test Data Management:

TDM is the process of creating, storing, managing, and maintaining the datasets used in the testing activity. The test data is the set of values of input used during the test process. It is used to secure sensitive information for test data to protect privacy. The process of creating smaller data from a larger dataset without affecting the internal settings is called data subsetting.

Generating new artificial synthetic data from real data is data generation. Identifying the data quality issues to resolving them before execution is called data profiling. Setting up the data to be ready to use the right data for the right case at the right time is known.

As data provisioning.

Features of TDM:-

1. **Data profiling:** It collects the data to check the data quality and issues that are in the data during the test.
2. **Data Transformation:** Modify data to fit specific testing. It involves the data cleaning, information update that is important for a particular analysis.
3. **Data Refresh:** It is the process of checking data is currently updated or not for making an accurate prediction.
4. **Data masking:** It is used to ensure that the data is protected from outside or unauthorized access to preserve data privacy.
5. **Data Validation:** This ensures that the data is accurate, complete, and follows the rules and policies.
6. **Data Usage Reports:** Create reports to track data utilization, detailing which data sets were used during particular test runs.
7. **Protection of Sensitive Data:** Ensure that sensitive data, including personal identifiers, financial information, and secret information, is properly safeguarded.

Evaluation Framework

1. Data Fidelity

Three core approaches

1) Statistical Comparison: Statistical measurement for each column type, distribution comparison b/w continuous & categorical data, and also compare real and synthetic data.
2) Relationship(): finding the correlation for feature to feature type, pattern preservation check.
3) Coverage assessment: value range completeness, category preservation.

2. Key metrics:

NUM_FLOAT_NORM **&** NUM_FLOAT_FREE for mean and std preservation.

2. Data Integrity

- **Checks**:

  Column-level: type, range, format

  Cross-column: keys, dependencies

  Business rules: logic, consistency

3. Data Utility

- **TSTR (Train Synthetic, Test Real)**: Train ML on synthetic and real data separately
- **Discriminator Method**: (Compare performance and feature importance)

Modeling with GANs

- **Generator**: Learns to synthesize realistic data, learn to distinguish real vs fake. No guaranteed feature interaction. Potential for manual feature interaction or better architectures.

GAN Models:

1. CTGAN (Conditional Tabular GAN)

- Designed for Tabular data with mixed data types (categorical + continuous**).**
- Maintains correlations between variables.

Note: Handles high-cardinality categorical features well

2. TVAE (Tabular Variational Autoencoder)

- Designed for Tabular data using probabilistic modelling.
- Trained to reconstruct data by learning a probability distribution over features.
- Handle both numerical and categorical variables.

Note: Stable training, Better for smaller datasets or when less noise tolerance is needed.

3. CTABGAN (Conditional Table GAN)

- Designed for: Better generalization across diverse tabular data.
- Combines CTGAN with additional preprocessing: log transform, binning, **etc.**

3. Introduction of Synthetic Data Generation (SDG)

3.3.Train Synthetic Test Real (TSTR) Methodology

TSTR method is commonly used evaluation tasks. It has ability to train effective ML models , provides an standard framework for evaluation.

TSTR Workflow:

To implement specific sequence of steps with the original & synthetic dataset

Let, Original dataset = R

Synthetic dataset = S

Step 1 : Data Partitioning

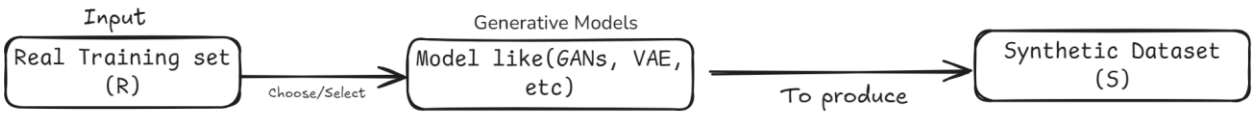
Splitting the available real dataset R into two sets

Training set = $R_{(train)}$, Creating/generating synthetic data or training set for baseline model used to predict the outcome of data.

Testing set = $R_{(test)}$, It is reserved for evaluating final models.(not used during model training or creation)

Step 2 : Synthetic Data Generation

Use real training set (R) as I/p to your chosen model to produce the synthetic dataset (S)



-Step 3 : Train on Synthetic Data

Select the suitable supervised ML model like logistic regression, random forest, SVM etc. Let Train model as (M) using the on;ly synthetic dataset (S) as its training data.

Step 4 : Test on Real Data

To evaluate the model MMM trained on synthetic data, we test it on the real test set $R_{(test}$. Performance is measured using standard metrics:

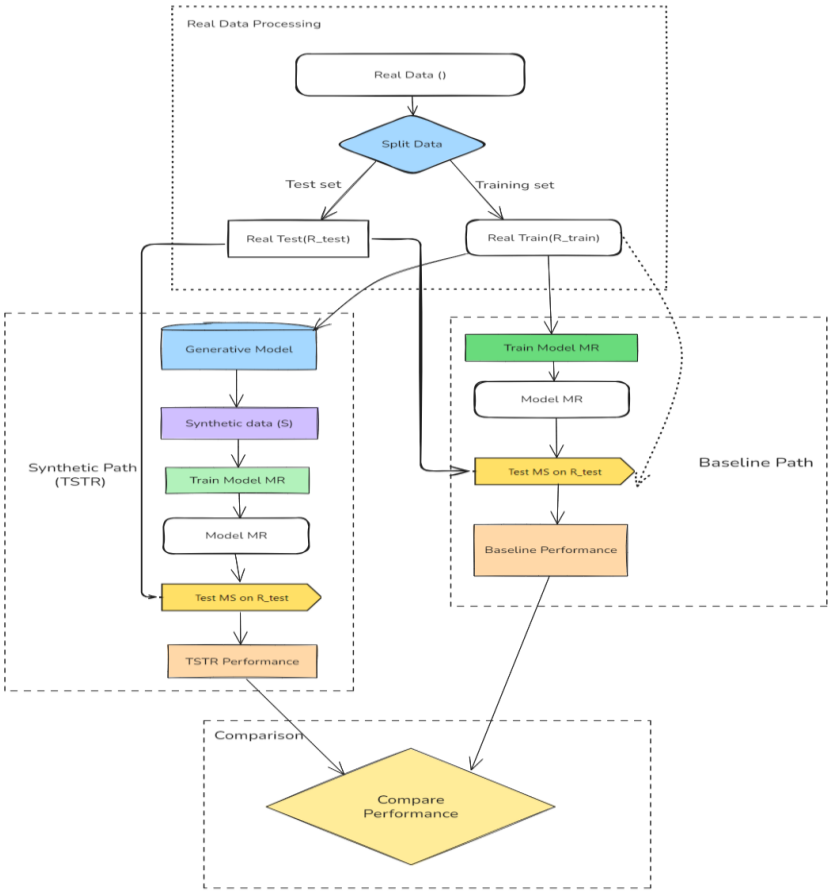
Classification: Accuracy, F1-score, AUC

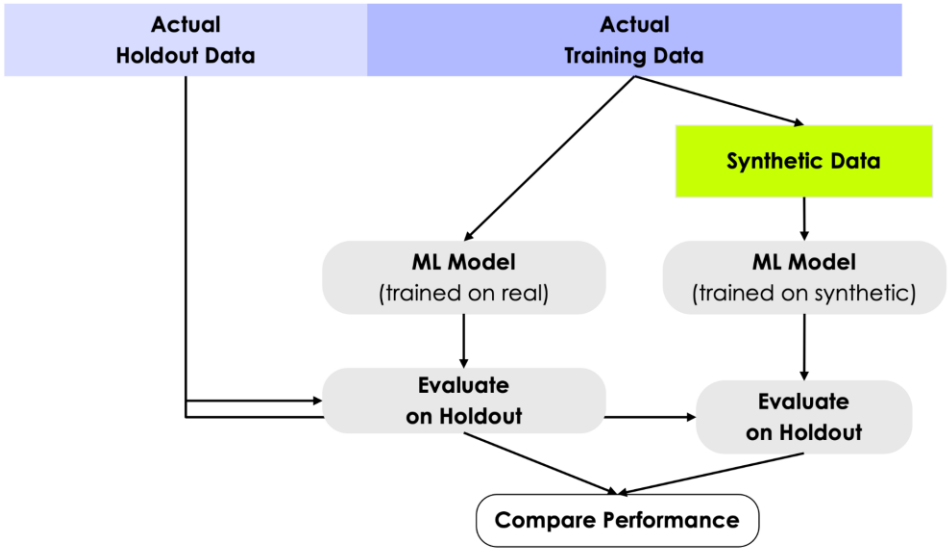
Regression: MAE, RMSE

This evaluation gives the TSTR performance score, showing how well the synthetic trained model performs on real data.

Step 5: Establish a Baseline

Train the same model (M) on real training data $R_{(train)}$ and evaluate on $R_{(test)}$. This gives a benchmark for ideal performance using original data.





SDV (Synthetic Data Vault)

It is a python library use for creating tabular synthetic data. It generates the new data based on pattern learned from original data. It supports multiple model like GAN, TAVE

Task-5: Implement Of the TSTR & explain how it work

1. Library Installation and Drive Mounting

You began with !pip install sdv to ensure that the SDV (Synthetic Data Vault) library was available in your Google Colab environment since SDV is essential for generating high quality synthetic data for data-driven model evaluation. By importing pandas and mounting Google Drive with drive.mount(), to ensured smooth data access and manipulation and made datasets stored in Google Drive accessible.

```
!pip install sdv

import pandas as pd
from google.colab import drive
drive.mount('/content/drive/')
```

2. Data Loading and Cleaning

Loaded the real health dataset using pd.read_csv() directly from Google Drive. Because machine learning models can be sensitive to inconsistencies, immediately cleaned the dataset by dropping duplicates and removing rows with missing values using drop_duplicates() and dropna(). This makes sure that both real data and any synthetic data generated later are reliable and do not have artifacts that could bias your results or cause errors when training models.

```
# Load the dataset
real_data = pd.read_csv("/content/drive/MyDrive/Durgesh(testing)/Code (TSTR)/health_disease_prediction.csv")
real_data.head()
#data cleaning
real_data = real_data.drop_duplicates()
real_data = real_data.dropna()
```

3. Label Encoding Categorical Data

To use the dataset with machine learning algorithms, needs all input features and the target column to be numeric. Many ML algorithms, including KNN and tree-based models, cannot handle string labels. Therefore, I used LabelEncoder from sklearn to convert the Disease column from text to integers, making it compatible with the ML models planned to use.

```
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
real_data['Disease'] = label_encoder.fit_transform(real_data['Disease'])
```

\

4. Metadata Detection for SDV

Before generating synthetic data, I have to explain to SDV the structure of dataset. By using Metadata.detect_from_dataframe, allow SDV to automatically infer the data types and relationships between the columns. This step is crucial because SDV's generative models require clear schema information to accurately mimic the data's structure and constraints, which helps in generating realistic synthetic samples.

```
from sdv.metadata import Metadata
metadata = Metadata.detect_from_dataframe(
    data=real_data,
    table_name='health_disease_prediction'
)
```

5. Synthetic Data Generation using TVAE

Choosing the TVAESynthesizer, a model based on neural network architectures, to generate synthetic data. This choice is motivated by TVAE's ability to preserve complex relationships between columns better than simple random sampling approaches. Training on the cleaned real data allowed the TVAE model to learn the with-in data distributions, and sampling 1500 synthetic records created a dataset large enough for robust testing and evaluation.

```
from sdv.single_table import TVAESynthesizer
synthesizer = TVAESynthesizer(metadata)
synthesizer.fit(real_data)
```

```
synthetic_data = synthesizer.sample(num_rows=1500)
```

6. Feature Splitting

To prepare for machine learning, I am splitting both the real and synthetic datasets into features (X) and the label (y). Dropping the target column into a separate variable is a standard step because ML algorithms are generally supervised they learn to predict y (target) from X (input features).

```
# split features & goal
X_synthetic = synthetic_data.drop(['Disease'], axis=1)
y_synthetic = synthetic_data['Disease']

X_real = real_data.drop(['Disease'], axis=1)
y_real = real_data['Disease']
```

7. Train-Test Splitting

I have applied train_test_split to split your datasets into training and testing subsets. Splitting is critical to measure how well a model generalizes: by training on one part of the data and testing on an unseen holdout. This also mimics real-world scenarios where models are deployed on new, unseen data, boosting the credibility of evaluation.

```
from sklearn.model_selection import train_test_split
X_train_synthetic,
 X_test_synthetic,
  y_train_synthetic,
   y_test_synthetic = train_test_split(X_synthetic,
                                       y_synthetic,
                                       test_size=0.3,
                                       random_state=42)
X_train_real,
 X_test_real,
  y_train_real,
   y_test_real = train_test_split(X_real, y_real, test_size=0.3, random_state=42)
```

8. Feature Scaling

Since KNN (and many ML algorithms) is sensitive to the scale of input features, I have normalized the features using StandardScaler (and experimented with MinMaxScaler). This transformation ensures that all features contribute equally to the distance-based calculations in KNN, rather than being dominated by variables with large numeric ranges.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_real = scaler.fit_transform(X_train_real)
X_test_real = scaler.transform(X_test_real)
X_train_synthetic = scaler.transform(X_train_synthetic)
X_test_synthetic = scaler.transform(X_test_synthetic)
```

9. KNN Training on Synthetic, Testing on Real (TSTR)

The core experiment was to train a KNN classifier **only on synthetic data** and test it on **real data**. This follows the TSTR (Train Synthetic, Test Real) paradigm, which is a powerful way of assessing whether the synthetic data captures real-world distributions well enough that a model can generalize from synthetic to real samples. If performance is strong, it means the synthetic data is of high fidelity and utility.

```
# KNN Model
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_synthetic, y_train_synthetic)
```

```
# Predict on real test data
y_pred_knn = knn_model.predict(X_test_real)
```

Output:-

```
KNN Classifier Results:
Accuracy: 0.70
Precision: 0.60
Recall: 0.70
F1 Score: 0.58
```

10. Classification Performance Evaluation

I used metrics such as accuracy, precision, recall, and F1 score. These metrics cover different performance perspectives: accuracy for overall correctness, precision for positive predictive value, recall for sensitivity, and F1 for a balance of the two. This comprehensive evaluation is important because it reveals potential weaknesses such a model that is accurate but fails to recall certain classes), especially in healthcare where errors can be costly.

```
# evaluation of KNN
print("\nKNN Classifier Results:")
print(f"Accuracy: {accuracy_score(y_test_real, y_pred_knn):.2f}")
print(f"Precision: {precision_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
print(f"Recall: {recall_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
print(f"F1 Score: {f1_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
# save to CSV
```

11.XGboost classifier & analysed its accuracy on real test data

```
# XGBoost classifier
!pip install xgboost
from xgboost import XGBClassifier

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
xgb_model.fit(X_train_synthetic, y_train_synthetic)

#Predict on real test data
y_pred_xgb = xgb_model.predict(X_test_real)

# Evaluation
print("\nXGBoost Classifier Results:")
print(f"Accuracy: {accuracy_score(y_test_real, y_pred_xgb):.2f}")
print(f"Precision: {precision_score(y_test_real, y_pred_xgb, average='weighted'):.2f}")
print(f"Recall: {recall_score(y_test_real, y_pred_xgb, average='weighted'):.2f}")
print(f"F1 Score: {f1_score(y_test_real, y_pred_xgb, average='weighted'):.2f}")
```

```
XGBoost Classifier Results:
Accuracy: 0.58
Precision: 0.58
Recall: 0.58
F1 Score: 0.58
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [07
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

12. Visualization: Confusion Matrix and Metric Bar Plot

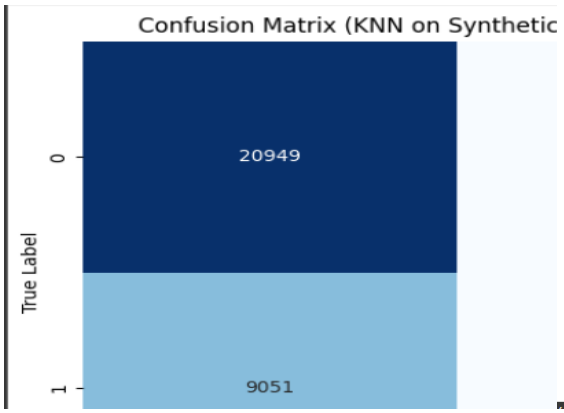
Visualization is essential for effective communication of ML results. The confusion matrix illustrates precisely how the model performs across different disease categories, making it easier to see where it struggles. The metrics bar plot succinctly compares performance metrics, allowing quick scanning and highlighting tradeoffs between accuracy, precision, recall, and F1.

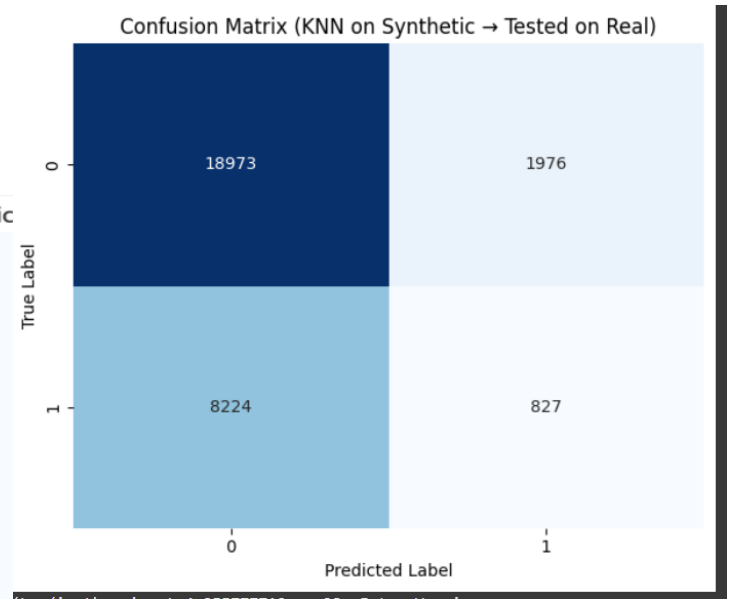
```
import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
```

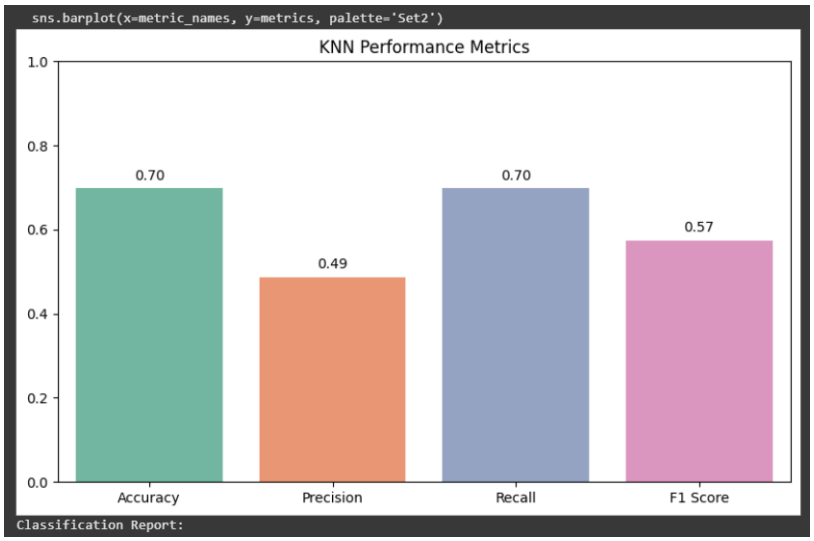
```
#Confusion Matrix Plot

cm = confusion_matrix(y_test_real, y_pred_knn)

plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,

            xticklabels=label_encoder.classes_,

            yticklabels=label_encoder.classes_)

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

plt.title('Confusion Matrix (KNN on Synthetic → Tested on Real)')

plt.tight_layout()

plt.show()

#bar plot of evaluation matrix

accuracy = accuracy_score(y_test_real, y_pred_knn)

precision = precision_score(y_test_real, y_pred_knn, average='weighted')

recall = recall_score(y_test_real, y_pred_knn, average='weighted')

f1 = f1_score(y_test_real, y_pred_knn, average='weighted')



metrics = [accuracy, precision, recall, f1]

metric_names = ['Accuracy', 'Precision', 'Recall', 'F1 Score']



plt.figure(figsize=(8, 5))

sns.barplot(x=metric_names, y=metrics, palette='Set2')

plt.ylim(0, 1)

plt.title('KNN Performance Metrics')

for i, val in enumerate(metrics):

    plt.text(i, val + 0.02, f"{val:.2f}", ha='center')

plt.tight_layout()
```

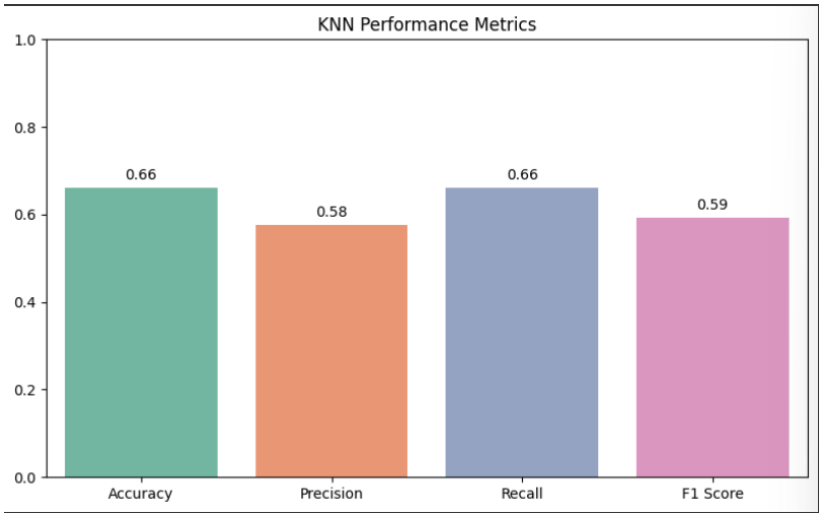
```
plt.show()
```

Output:









13. Final Hyperparameter Sweep with MinMaxScaler

Finally, I repeated the KNN experiment with MinMaxScaler and visualized KNN's performance across different k values to identify the best number of neighbors. This sweep helps identify robust patterns and the most effective settings for use-case, ensuring that the conclusions drawn are not artefacts of a poorly-chosen hyperparameter.

Preprocessing techniques in knn

```
# MinmaxScaler
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_train_real = scaler.fit_transform(X_train_real)
X_test_real = scaler.transform(X_test_real)
X_train_synthetic = scaler.transform(X_train_synthetic)
X_test_synthetic = scaler.transform(X_test_synthetic)

# Hyperparameter tuning
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

k_values = range(1, 16)
scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_synthetic, y_train_synthetic)
    y_pred = knn.predict(X_test_real)

    acc = accuracy_score(y_test_real, y_pred)
    f1 = f1_score(y_test_real, y_pred, average='weighted')
    scores.append((k, acc, f1))

# Convert to df for  plotting
scores_df = pd.DataFrame(scores, columns=['k', 'Accuracy', 'F1 Score'])
```

14. Comparison with Gradient Boosting

I also trained a more advanced model (Gradient Boosting) on synthetic data and tested it on real data. The goal here was to check whether conclusions about the quality of synthetic data hold across different types of models: simple (KNN) and complex (ensemble trees). Gradient Boosting can often detect subtler patterns which KNN might miss, using it validates that the synthetic data's utility is not specific to one model.

Gradient boosting vs KNN

```
[4] import pandas as pd
    import numpy as np
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score, classification_report
    from sklearn.preprocessing import LabelEncoder, StandardScaler
    from sklearn.ensemble import GradientBoostingClassifier
    from sklearn.neighbors import KNeighborsClassifier

    #loading real and synthetic data
    real_data = pd.read_csv(
        '/content/health_disease_prediction.csv')
    # Load the synthetic data generated and saved in the previous cell
    synthetic_data = pd.read_csv('/content/drive/MyDrive/Durgesh(testing)/synthetic_data_TVAE.csv')

    #preprocessing is already done in the previous cell, so we can use the existing dataframes
    # real_data, real_encoders = preprocess(real_data)
    # synthetic_data, _ = preprocess(synthetic_data)

    # Step 3: Separate Features and Target
    target_column = 'Disease'

    X_real = real_data.drop(columns=[target_column])
    y_real = real_data[target_column]

    X_synth = synthetic_data.drop(columns=[target_column])
    y_synth = synthetic_data[target_column]

    # Step 4: Train-Test Split (Test set is from real data)
    X_train_synth, y_train_synth = X_synth, y_synth
    X_test_real, y_test_real = X_real, y_real

    # Optional: Scaling for KNN
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train_synth)
    X_test_scaled = scaler.transform(X_test_real)

    #Train gradient boosting classifier on synthetic data
    gb_model = GradientBoostingClassifier()
    gb_model.fit(X_train_synth, y_train_synth)
    y_pred_gb = gb_model.predict(X_test_real)

    #train KNN classifier on synthetic Data
    knn_model = KNeighborsClassifier(n_neighbors=5)
    knn_model.fit(X_train_scaled, y_train_synth)
    y_pred_knn = knn_model.predict(X_test_scaled)

    # Step 7: Evaluate and Compare
    print("TSTR Performance - Gradient Boosting Classifier:")
    print("Accuracy:", accuracy_score(y_test_real, y_pred_gb))
    print(classification_report(y_test_real, y_pred_gb))

    print("\n TSTR Performance - K-Nearest Neighbors:")
    print("Accuracy:", accuracy_score(y_test_real, y_pred_knn))
    print(classification_report(y_test_real, y_pred_knn))
```

Output:-

```
TSTR Performance - Gradient Boosting Classifier:
Accuracy: 0.67674
              precision    recall  f1-score   support

           0       0.70      0.94      0.80     70075
           1       0.29      0.06      0.10     29925

    accuracy                           0.68    100000
   macro avg       0.50      0.50      0.45    100000
weighted avg       0.58      0.68      0.59    100000


 TSTR Performance - K-Nearest Neighbors:
Accuracy: 0.67808
              precision    recall  f1-score   support

           0       0.70      0.94      0.80     70075
           1       0.30      0.06      0.09     29925

    accuracy                           0.68    100000
   macro avg       0.50      0.50      0.45    100000
weighted avg       0.58      0.68      0.59    100000
```

15.KNN Hyperparameter Tuning

To optimize your KNN performance, I have systematically varied the number of neighbors (k) and the distance metric. This grid search approach is a standard ML practice to discover the most suitable hyperparameters for a given scenario, as KNN's performance is highly sensitive to these choices

```
import matplotlib.pyplot as plt

# Define parameter grid
neighbors = [3, 5, 7, 9]
metrics = ['euclidean', 'manhattan', 'minkowski']

# Store results
results = []

print("Tuning KNN with different neighbors and distance metrics \n")

# Iteration
for k in neighbors:
    for metric in metrics:
        knn_model = KNeighborsClassifier(n_neighbors=k, metric=metric)
        knn_model.fit(X_train_synthetic, y_train_synthetic)
        y_pred = knn_model.predict(X_test_real)

        acc = accuracy_score(y_test_real, y_pred)
        prec = precision_score(y_test_real, y_pred, average='weighted')
        rec = recall_score(y_test_real, y_pred, average='weighted')
        f1 = f1_score(y_test_real, y_pred, average='weighted')

        print(f"k={k}, metric={metric} → Acc: {acc:.2f}, Prec: {prec:.2f}, Rec: {rec:.2f}, F1: {f1:.2f}")
        results.append({'k': k, 'metric': metric, 'accuracy': acc, 'precision': prec, 'recall': rec, 'f1_score': f1})
```

Output:

```
Tuning KNN with different neighbors and distance metrics

k=3, metric=euclidean → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=3, metric=manhattan → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=3, metric=minkowski → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
k=5, metric=euclidean → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=5, metric=manhattan → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=5, metric=minkowski → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
k=7, metric=euclidean → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=7, metric=manhattan → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=7, metric=minkowski → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
k=9, metric=euclidean → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=9, metric=manhattan → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
k=9, metric=minkowski → Acc: 0.70, Prec: 0.49, Rec: 0.70, F1: 0.57
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

16. Baseline Evaluation: KNN Model Trained and Tested on Real Data (Real-to-Real)

To establish a solid benchmark for comparison with TSTR results, I have done Real to Real evaluation of the KNN classifier. In this, the model was both trained and tested on real-world health disease prediction data using a train test split. Before training, the data was cleaned by removing duplicates and missing values, followed by label encoding of the categorical target feature such as Disease. Numerical features were scaled using StandardScaler to ensure fair distance-based computations in the KNN model.

I have trained the KNN classifier using default hyperparameters (n_neighbors=5) and evaluated its performance on the held-out real test data using standard metrics including Accuracy, Precision, Recall, and F1 score

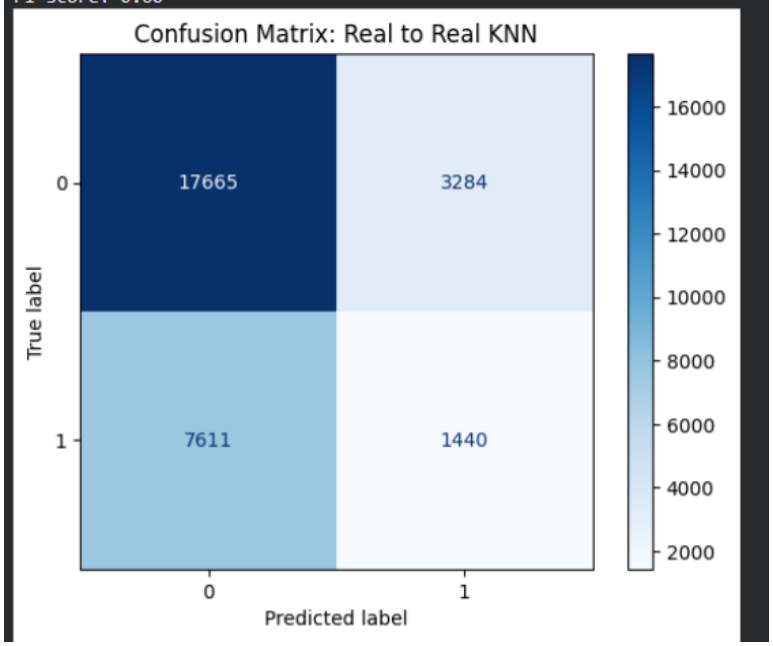
Real-to-Real KNN Baseline for comparison with TSRT

Evaluating the KNN model on real data to establish baseline for comparison with TSTR outpu

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
#train KNN on real-to-real data
knn_real = KNeighborsClassifier(n_neighbors=5)
knn_real.fit(X_train_real, y_train_real)

#predict on real_data
y_pred_real = knn_real.predict(X_test_real)
#Evaluation
real_accuracy = accuracy_score(y_test_real, y_pred_real)
real_precision = precision_score(y_test_real, y_pred_real, average='weighted')
real_recall = recall_score(y_test_real, y_pred_real, average='weighted')
real_f1 = f1_score(y_test_real, y_pred_real, average='weighted')

print("\n Real to Real KNN Results:")
print(f"Accuracy: {real_accuracy:.2f}")
print(f"Precision: {real_precision:.2f}")
print(f"Recall: {real_recall:.2f}")
print(f"F1 score: {real_f1:.2f}")
# Confusion Matrix
cm = confusion_matrix(y_test_real, y_pred_real)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=label_encoder.classes_)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix: Real to Real KNN")
plt.show()


# compare with the tstr  results
tstr_accuracy = accuracy_score(y_test_real, y_pred_knn)
tstr_precision = precision_score(y_test_real, y_pred_knn, average='weighted')
tstr_recall = recall_score(y_test_real, y_pred_knn, average='weighted')
tstr_f1 = f1_score(y_test_real, y_pred_knn, average='weighted')
```

Output:

```
 Real to Real KNN Results:
Accuracy: 0.64
Precision: 0.58
Recall: 0.64
F1 score: 0.60
```

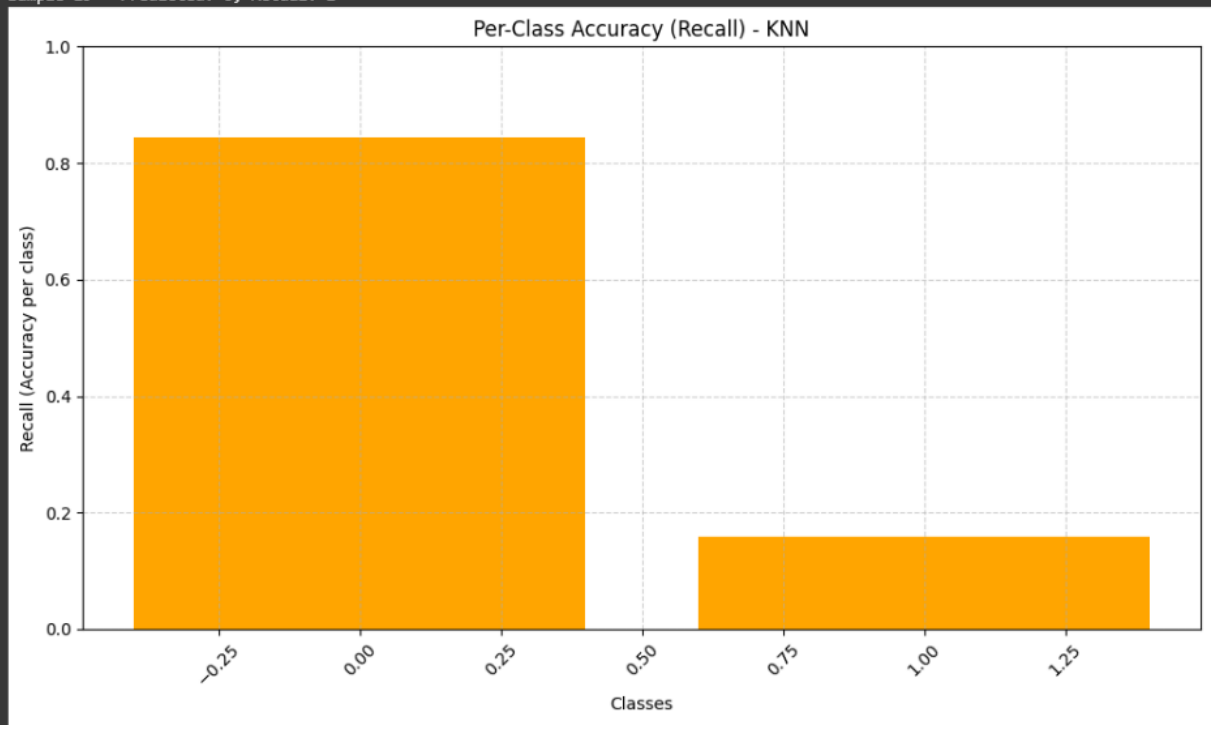


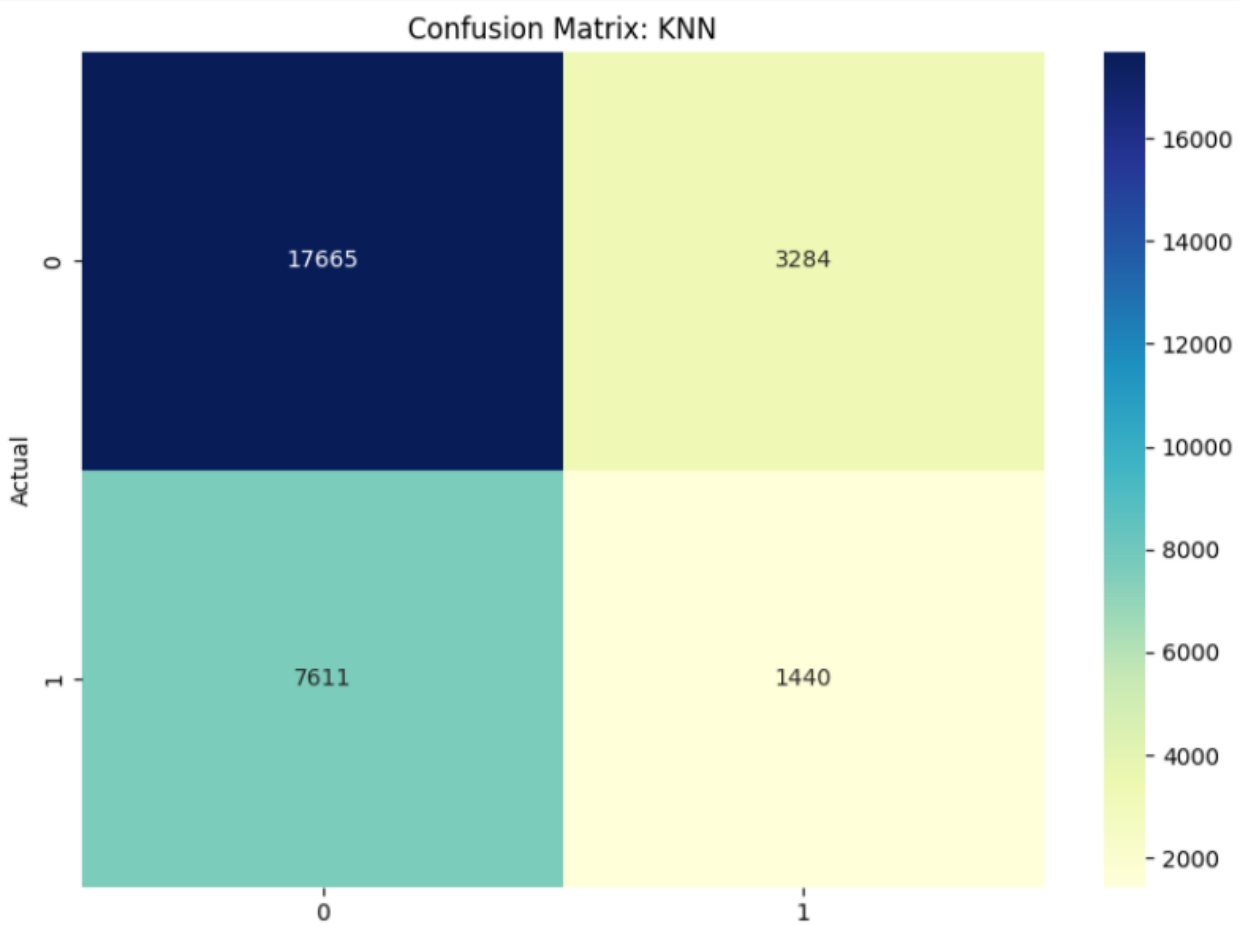
17. Error Analysis

```python
# 1. Preprocess
real_data = real_data.drop_duplicates().dropna()
label_encoder = LabelEncoder()
real_data['Disease'] = label_encoder.fit_transform(real_data['Disease'])
X = real_data.drop(['Disease'], axis=1)
y = real_data['Disease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train on real_data (Real to Real baseline)
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)

#2. Extract misclassified index
misclassified_idx = np.where(y_test.values != y_pred_knn)[0]

print("\nMisclassified Examples (first 5):")
for i in misclassified_idx[:5]:
    print(f"Sample {i} → Predicted: {y_pred_knn[i]}, Actual: {y_test.values[i]}")

#3. Per-class accuracy
report = classification_report(y_test, y_pred_knn, target_names=label_encoder.classes_, output_dict=True)
classes = list(report.keys())[:-3]
recalls = [report[cls]['recall'] for cls in classes]

plt.figure(figsize=(10, 6))
plt.bar(classes, recalls, color='orange')
plt.title("Per-Class Accuracy (Recall) - KNN")
plt.xlabel("Classes")
plt.ylabel("Recall (Accuracy per class)")
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()

#Enhancing Confusion Matrix
cm = confusion_matrix(y_test, y_pred_knn)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap="YlGnBu",
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix: KNN")
plt.tight_layout()
plt.show()
```

Output:

```
Misclassified Examples (first 5):
Sample 3 → Predicted: 0, Actual: 1
Sample 7 → Predicted: 0, Actual: 1
Sample 8 → Predicted: 0, Actual: 1
Sample 18 → Predicted: 0, Actual: 1
Sample 19 → Predicted: 0, Actual: 1
```





Task-6: TSTR using techniques & hyperparameters.ipynb

2) TSTR using techniques & hyperparameters.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
!pip install sdv

import pandas as pd
from google.colab import drive
import os

# Load the dataset
real_data = pd.read_csv("/content/health_disease_prediction.csv")
real_data.head()
#data cleaning
real_data = real_data.drop_duplicates()
real_data = real_data.dropna()

# Label encode categorical columns including goal
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
real_data['Disease'] = label_encoder.fit_transform(real_data['Disease'])  # Target column


#metadata
from sdv.metadata import Metadata
metadata = Metadata.detect_from_dataframe(
    data=real_data,
    table_name='health_disease_prediction'
)

# train TVAE synthesizer
from sdv.single_table import TVAESynthesizer
synthesizer = TVAESynthesizer(metadata)
synthesizer.fit(real_data)

#generate synthetic data
synthetic_data = synthesizer.sample(num_rows=1500)

# split features & goal
X_synthetic = synthetic_data.drop(['Disease'], axis=1)
y_synthetic = synthetic_data['Disease']

X_real = real_data.drop(['Disease'], axis=1)
y_real = real_data['Disease']

# train test split
from sklearn.model_selection import train_test_split
X_train_synthetic, X_test_synthetic, y_train_synthetic, y_test_synthetic = train_test_split(X_synthetic,
                                        y_synthetic,
                                        test_size=0.3,
                                        random_state=42)
X_train_real, X_test_real, y_train_real, y_test_real = train_test_split(X_real, y_real, test_size=0.3, random_state=42)

# feature scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_real = scaler.fit_transform(X_train_real)
X_test_real = scaler.transform(X_test_real)
X_train_synthetic = scaler.transform(X_train_synthetic)
X_test_synthetic = scaler.transform(X_test_synthetic)

# KNN Model
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_synthetic, y_train_synthetic)

# Predict on real test data
y_pred_knn = knn_model.predict(X_test_real)

# evaluation of KNN
print("\nKNN Classifier Results:")
print(f"Accuracy: {accuracy_score(y_test_real, y_pred_knn):.2f}")
print(f"Precision: {precision_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
print(f"Recall: {recall_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
print(f"F1 Score: {f1_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
# save to CSV
output_dir = '/content/drive/MyDrive/Durgesh(testing)'
os.makedirs(output_dir, exist_ok=True)
synthetic_data.to_csv(f'{output_dir}/synthetic_data_TVAE.csv', index=False)
```

2) TSTR using techniques & hyperparameters.ipynb

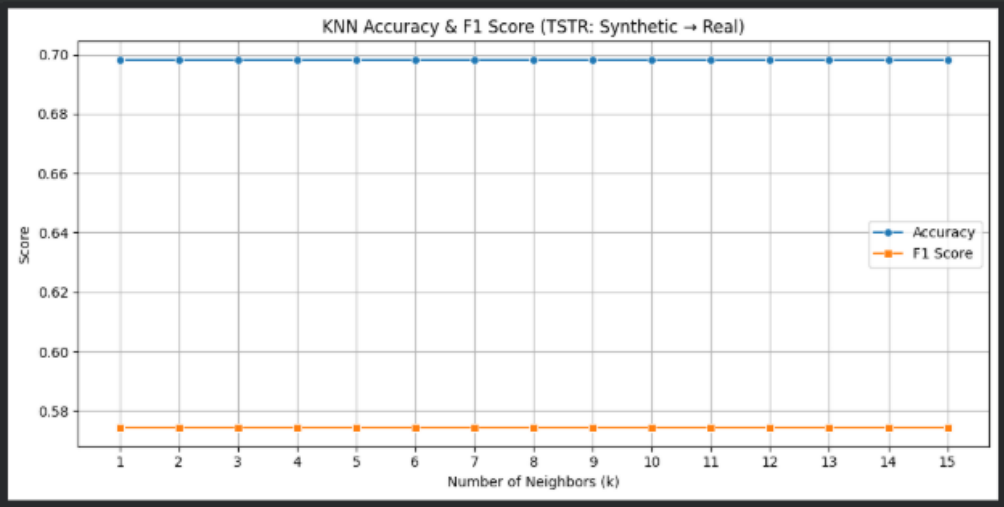
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Preprocessing techniques in knn

```python
# Hyperparameter tuning
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

k_values = range(1, 16)
scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_synthetic, y_train_synthetic)
    y_pred = knn.predict(X_test_real)

    acc = accuracy_score(y_test_real, y_pred)
    f1 = f1_score(y_test_real, y_pred, average='weighted')
    scores.append((k, acc, f1))

# Convert to df for  plotting
scores_df = pd.DataFrame(scores, columns=['k', 'Accuracy', 'F1 Score'])
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 5))
sns.lineplot(x='k', y='Accuracy', data=scores_df, marker='o', label='Accuracy')
sns.lineplot(x='k', y='F1 Score', data=scores_df, marker='s', label='F1 Score')
plt.title('KNN Accuracy & F1 Score (TSTR: Synthetic → Real)'\n 'Hyperparameter Tuning')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Score')
plt.xticks(k_values)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Task-7: TSTR Gradient Boosting & KNN Comparison

3) TSTR Gradient boost vs KNN

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import pandas as pd
from google.colab import drive

# Load the dataset
real_data = pd.read_csv("/content/health_disease_prediction.csv")
real_data.head()
#data cleaning
real_data = real_data.drop_duplicates()
real_data = real_data.dropna()

# Label encode categorical columns including the target
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
real_data['Disease'] = label_encoder.fit_transform(real_data['Disease'])


# Metadata
from sdv.metadata import Metadata
metadata = Metadata.detect_from_dataframe(
    data=real_data,
    table_name='health_disease_prediction'
)

# Train TVAE Synthesizer
from sdv.single_table import TVAESynthesizer
synthesizer = TVAESynthesizer(metadata)
synthesizer.fit(real_data)

#generate synthetic data
synthetic_data = synthesizer.sample(num_rows=1500)

# split features & goal
X_synthetic = synthetic_data.drop(['Disease'], axis=1)
y_synthetic = synthetic_data['Disease']

X_real = real_data.drop(['Disease'], axis=1)
y_real = real_data['Disease']

# Train-test split
from sklearn.model_selection import train_test_split
X_train_synthetic, X_test_synthetic, y_train_synthetic, y_test_synthetic = train_test_split(X_synthetic, y_synthetic, test_size=0.3, random_state=42)
X_train_real, X_test_real, y_train_real, y_test_real = train_test_split(X_real, y_real, test_size=0.3, random_state=42)

# Feature scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_real = scaler.fit_transform(X_train_real)
X_test_real = scaler.transform(X_test_real)
X_train_synthetic = scaler.transform(X_train_synthetic)
X_test_synthetic = scaler.transform(X_test_synthetic)

# KNN Model
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_synthetic, y_train_synthetic)

# Predict on real test data
y_pred_knn = knn_model.predict(X_test_real)

# Evaluation
print("\nKNN Classifier Results:")
print(f"Accuracy: {accuracy_score(y_test_real, y_pred_knn):.2f}")
print(f"Precision: {precision_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
print(f"Recall: {recall_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
print(f"F1 Score: {f1_score(y_test_real, y_pred_knn, average='weighted'):.2f}")
# Save it to CSV
synthetic_data.to_csv('/content/drive/MyDrive/Durgesh(testing)/synthetic_data_TVAE.csv', index=False)
```

3) TSTR Gradient boost vs KNN

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
KNN Classifier Results:
Accuracy: 0.70
Precision: 0.60
Recall: 0.70
F1 Score: 0.58
```

Gradient boosting vs KNN

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier

#loading real and synthetic data
real_data = pd.read_csv(
    '/content/health_disease_prediction.csv')
# Load the synthetic data generated and saved in the previous cell
synthetic_data = pd.read_csv('/content/drive/MyDrive/Durgesh(testing)/synthetic_data_TVAE.csv')

#preprocessing is already done in the previous cell, so we can use the existing dataframes
# real_data, real_encoders = preprocess(real_data)
# synthetic_data, _ = preprocess(synthetic_data)

# Step 3: Separate Features and Target
target_column = 'Disease'

X_real = real_data.drop(columns=[target_column])
y_real = real_data[target_column]

X_synth = synthetic_data.drop(columns=[target_column])
y_synth = synthetic_data[target_column]

# Step 4: Train-Test Split (Test set is from real data)
X_train_synth, y_train_synth = X_synth, y_synth
X_test_real, y_test_real = X_real, y_real

# Optional: Scaling for KNN
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_synth)
X_test_scaled = scaler.transform(X_test_real)

#Train gradient boosting classifier on synthetic data
gb_model = GradientBoostingClassifier()
gb_model.fit(X_train_synth, y_train_synth)
y_pred_gb = gb_model.predict(X_test_real)

#train KNN classifier on synthetic Data
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_scaled, y_train_synth)
y_pred_knn = knn_model.predict(X_test_scaled)

# Step 7: Evaluate and Compare
print("TSTR Performance - Gradient Boosting Classifier:")
print("Accuracy:", accuracy_score(y_test_real, y_pred_gb))
print(classification_report(y_test_real, y_pred_gb))

print("\n TSTR Performance - K-Nearest Neighbors:")
print("Accuracy:", accuracy_score(y_test_real, y_pred_knn))
print(classification_report(y_test_real, y_pred_knn))
```

```
TSTR Performance - Gradient Boosting Classifier:
Accuracy: 0.67674
              precision    recall  f1-score   support

           0       0.70      0.94      0.80     70075
           1       0.29      0.06      0.10     29925

    accuracy                           0.68    100000
   macro avg       0.50      0.50      0.45    100000
weighted avg       0.58      0.68      0.59    100000


 TSTR Performance - K-Nearest Neighbors:
Accuracy: 0.67888
              precision    recall  f1-score   support

           0       0.70      0.94      0.80     70075
           1       0.30      0.06      0.09     29925

    accuracy                           0.68    100000
   macro avg       0.50      0.50      0.45    100000
weighted avg       0.58      0.68      0.59    100000
```

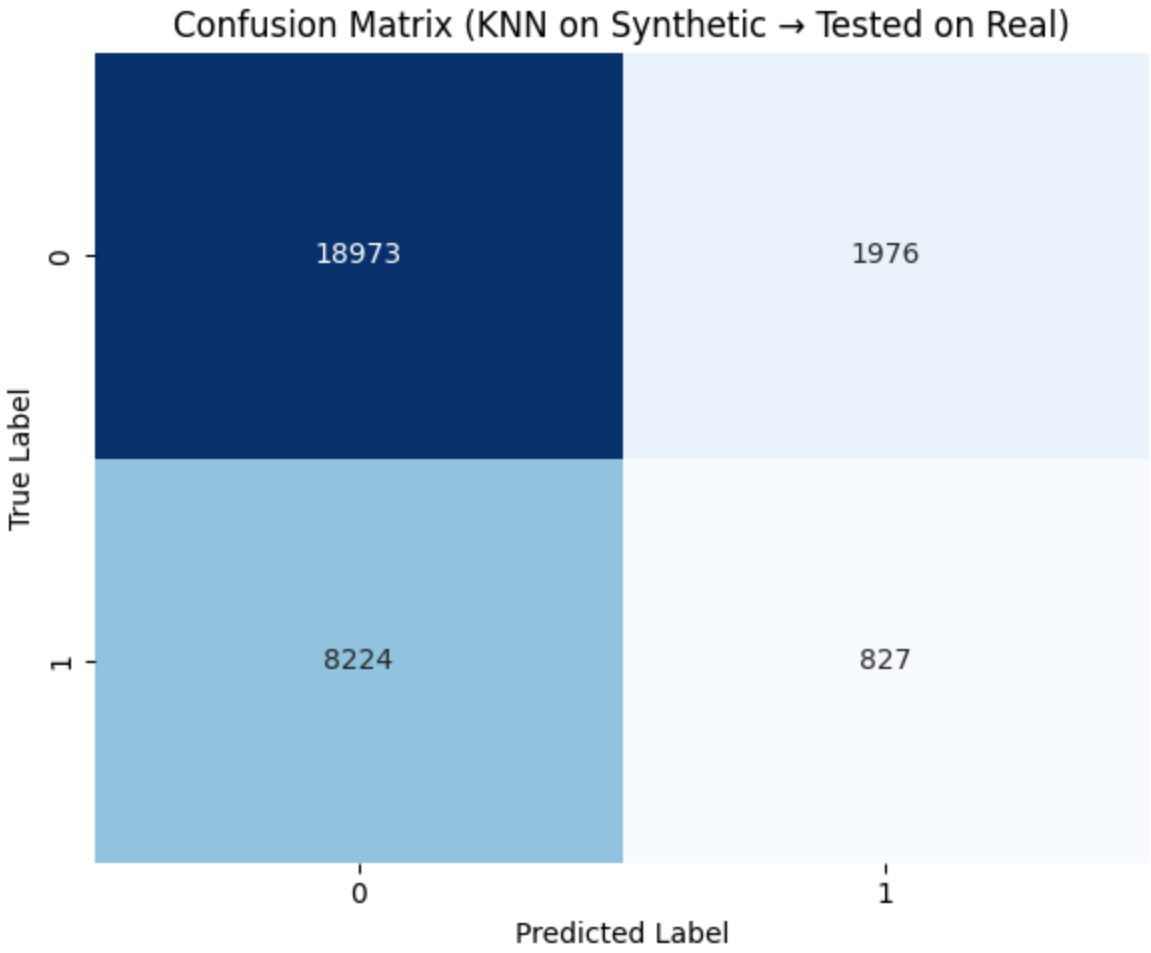
Task-8: XGBoost classifier & analyze it accuracy

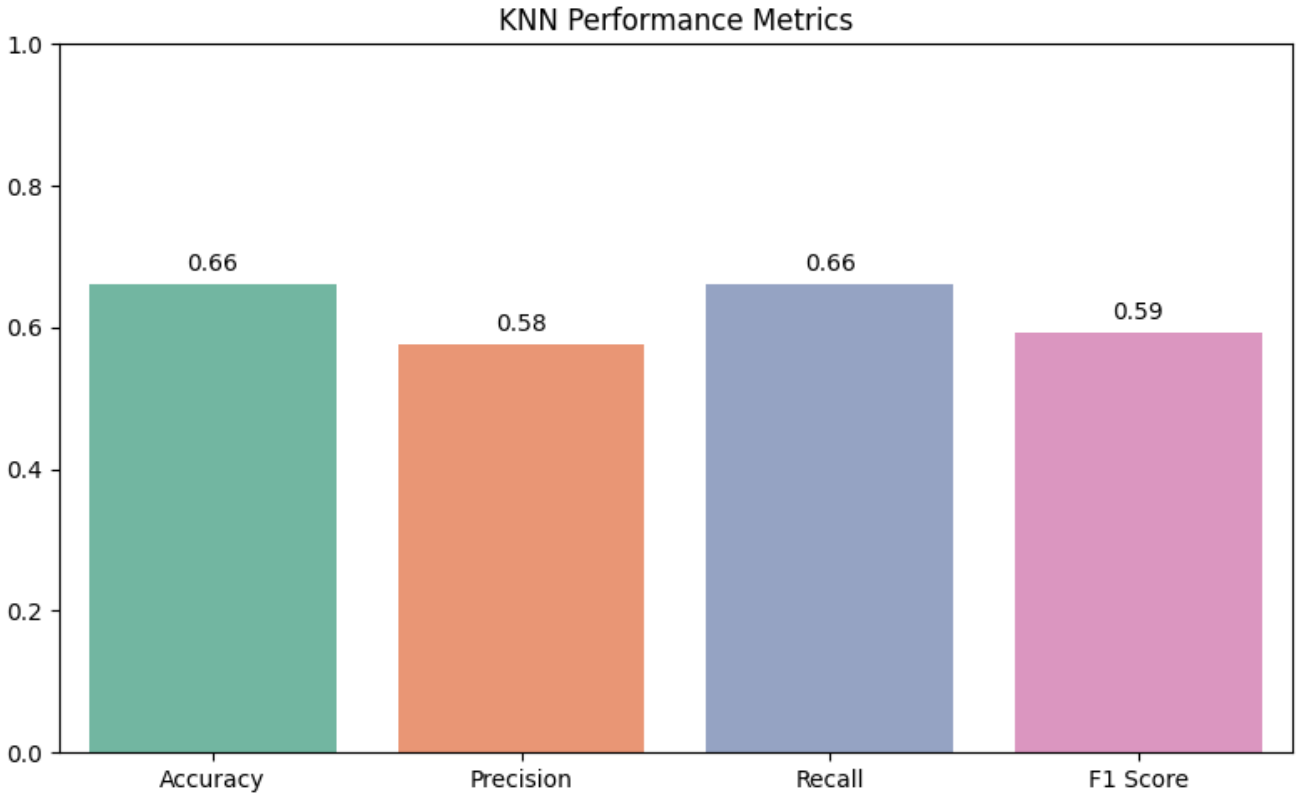
XGboost classifier & analysed its accuracy on real test data

```python
# XGBoost classifier
!pip install xgboost
from xgboost import XGBClassifier

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
xgb_model.fit(X_train_synthetic, y_train_synthetic)

#Predict on real test data
y_pred_xgb = xgb_model.predict(X_test_real)

# Evaluation
print("\nXGBoost Classifier Results:")
print(f"Accuracy: {accuracy_score(y_test_real, y_pred_xgb):.2f}")
print(f"Precision: {precision_score(y_test_real, y_pred_xgb, average='weighted'):.2f}")
print(f"Recall: {recall_score(y_test_real, y_pred_xgb, average='weighted'):.2f}")
print(f"F1 Score: {f1_score(y_test_real, y_pred_xgb, average='weighted'):.2f}")
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.11/dist-packages (3.0.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.11/dist-packages (from xgboost) (2.21.5)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from xgboost) (1.16.0)

XGBoost Classifier Results:
Accuracy: 0.58
Precision: 0.58
Recall: 0.58
F1 Score: 0.58
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [07:18:29] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

#Confusion Matrix Plot
cm = confusion_matrix(y_test_real, y_pred_knn)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix (KNN on Synthetic → Tested on Real)')
plt.tight_layout()
plt.show()


#bar plot of evaluation matrix
accuracy = accuracy_score(y_test_real, y_pred_knn)
precision = precision_score(y_test_real, y_pred_knn, average='weighted')
recall = recall_score(y_test_real, y_pred_knn, average='weighted')
f1 = f1_score(y_test_real, y_pred_knn, average='weighted')

metrics = [accuracy, precision, recall, f1]
metric_names = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

plt.figure(figsize=(8, 5))
sns.barplot(x=metric_names, y=metrics, palette='Set2')
plt.ylim(0, 1)
plt.title('KNN Performance Metrics')
for i, val in enumerate(metrics):
    plt.text(i, val + 0.02, f"{val:.2f}", ha='center')
plt.tight_layout()
plt.show()
```


Confusion Matrix (KNN on Synthetic → Tested on Real)
18973
1976
8224
827
True Label
0
1
0
1
Predicted Label


KNN Performance Metrics
1.0
0.8
0.6
0.4
0.2
0.0
0.66
0.58
0.66
0.59
Accuracy
Precision
Recall
F1 Score

Annexure I

testAIng.com
THE I IN AI

Thursday, 31 July 2025

Ref: Certificate of Internship Training/0511/Intern on Machine Learning

TO WHOM SO EVER IT MAY CONCERN

This is to certify that Mr. Durgesh Kumar Dewangan is undertaking internship training with us from 21st May 2025 to 30th July 2025.

Durgesh approached us for specialised training in field of AI and ML Technologies with us.

During internship he worked on synthetic data generation under the TDM_pipeline and applied TSTR methodology for model evaluation. He implemented and analyzed ML classifier models using real & synthetic datasets.

We wish him all the success in life.

for **testAIng solutions pvt.ltd. (tAI)**

Rakesh Nayar
Niti (Policy)

testAIng Solutions Pvt. Ltd.

1,Gurjar Mohalla, Near Arya Samaj School, old jail Road, Bikaner-334 005(Raj.) India.
Phone: 0151- 2544 333
GSTIN: 08AAHCT3799F1ZU | CIN: U72900RJ2019PTC065313